



Ngân hàng Hệ Điều Hành PTIT

Hệ điều hành (Học viện Công nghệ Bưu chính Viễn thông)



Scan to open on Studeersnel

Câu hỏi loại 1 điểm

Câu hỏi 1.1: Trình bày về giao diện lập trình của hệ điều hành?

Để các chương trình có thể sử dụng được những dịch vụ, HDH cung cấp giao diện lập trình.

- Giao diện này bao gồm các lời gọi hệ thống (system call) mà chương trình sử dụng yêu cầu một dịch vụ nào đó từ phía HDH.
- Lời gọi hệ thống: các lệnh đặc biệt mà CTUD gọi khi cần yêu cầu HDH thực hiện một việc gì đó.
- Lời gọi hệ thống được thực hiện qua những thư viện hàm gọi là thư viện hệ thống. Các hàm này sẽ giúp người lập trình gọi lời gọi hệ thống tương ứng của hệ điều hành

Câu hỏi 1.2: Trình bày kỹ thuật xử lý theo mẻ (lô) và ưu điểm của kỹ thuật này. Hệ thống xử lý theo mẻ có cần hệ điều hành không ?

Xử lý theo mẻ:

- Chương trình được phân thành các mẻ: gồm những chương trình có yêu cầu giống nhau
- Toàn bộ mẻ được nạp vào băng từ và được tải vào máy để thực hiện lần lượt

Để có thể tự động hóa xử lý theo mẻ, một chương trình nhỏ gọi là chương trình giám sát (monitor) được giữ thường xuyên trong bộ nhớ: mỗi khi một chương trình của mẻ kết thúc, chương trình giám sát tự động nạp chương trình tiếp theo vào máy và cho phép nó chạy

Ưu điểm của xử lý theo mẻ:

Giảm đáng kể thời gian chuyển đổi giữa hai chương trình trong cùng một mẻ => Tăng hiệu suất sử dụng CPU.

Hệ thống xử lý theo mẻ vẫn cần có hệ điều hành. Thực tế, chương trình giám sát (monitor) chính là dạng đơn giản nhất của hệ điều hành. Nó được tải vào và nằm thường trực trong bộ nhớ để quản lý việc thực hiện các chương trình của một mẻ.

Câu hỏi 1.3: Đa chương trình là gì ? Lý do sử dụng đa chương trình trong máy tính ? Yêu cầu đối với phần cứng khi sử dụng đa chương trình?

Đa chương trình:

- Hệ thống chứa đồng thời nhiều chương trình trong bộ nhớ
- Khi một chương trình phải dừng lại để thực hiện vào ra, HDH sẽ chuyển CPU sang thực hiện một chương trình khác

Lý do sử dụng đa chương trình:

Giảm thời gian chạy không tải của CPU => tăng hiệu suất sử dụng CPU.

Yêu cầu đối với phần cứng khi sử dụng đa chương trình:

Phải có hỗ trợ từ phía phần cứng, đặc biệt là khả năng vào/ra bằng ngắt và cơ chế DMA (Direct Memory Access). Nếu không có những cơ chế này, CPU sẽ phải trực tiếp điều khiển quá trình vào/ra thông tin và dữ liệu.

Câu hỏi 1.4: Trình bày về các thành phần của hệ thống máy tính và vai trò của hệ điều hành trong đó

- Phần cứng: cung cấp các tài nguyên cần thiết cho việc tính toán, xử lý dữ liệu
- Phần mềm: các chương trình cụ thể. (phần mềm hệ thống và phần mềm ứng dụng)
- HĐH: phần mềm đóng vai trò trung gian giữa phần cứng và người sử dụng chương trình ứng dụng, làm cho việc sử dụng hệ thống máy tính được tiện lợi và hiệu quả

Câu hỏi 1.5: Trình bày về hệ điều hành chia sẻ thời gian

Chia sẻ thời gian (đa nhiệm):

- Chia sẻ thời gian có thể coi như đa chương trình cải tiến
- CPU lần lượt thực hiện các công việc khác nhau trong những khoảng thời gian ngắn gọi là lượng tử thời gian
- Chuyển đổi giữa các công việc diễn ra với tần số cao và tốc độ CPU lớn

Tất cả người dùng đều có cảm giác máy tính chỉ thực hiện chương trình của mình

CPU được chia sẻ giữa những người dùng khác nhau tương tác trực tiếp với hệ thống

Câu hỏi 1.6 : Trình bày khái niệm tiến trình và chỉ rõ điểm khác nhau giữa tiến trình với chương trình. Nêu tên ít nhất bốn thao tác liên quan tới quản lý tiến trình (chỉ cần nêu tên, không cần trình bày chi tiết).

Tiến trình là một chương trình đang trong quá trình thực hiện.

Chương trình	Tiến trình
Thực thể tĩnh	Thực thể động
Không sở hữu tài nguyên cụ thể	Được cấp một số tài nguyên để chứa tiến trình và thực hiện lệnh

Thao tác liên quan tới quản lý tiến trình:

- Tạo mới tiến trình
- Kết thúc tiến trình
- Chuyển đổi giữa các tiến trình
- Điều độ tiến trình

Câu hỏi 1.7: Trình bày về thao tác tạo mới tiến trình. Tiến trình có thể bị kết thúc trong những trường hợp nào ?

Tạo mới tiến trình: Để tạo ra một tiến trình mới, OS thực hiện một số bước như sau:

- Gán số định danh cho tiến trình được tạo mới và tạo một ô trong bảng tiến trình
- Tạo không gian nhớ cho tiến trình và PCB
- Khởi tạo PCB: HĐH gán giá trị các thành phần của PCB
- Liên kết PCB của tiến trình vào các danh sách quản lý, ví dụ vào ds tiến trình đang chạy

Tiến trình bị kết thúc trong các trường hợp sau:

- Bị tiến trình cha kết thúc

- Do các lỗi
- Yêu cầu nhiều bộ nhớ hơn so với số lượng hệ thống có thể cung cấp
- Thực hiện lâu hơn thời gian giới hạn
- Do quản trị hệ thống hoặc hệ điều hành kết thúc

Câu hỏi 1.8: Thế nào là dòng (thread) mức người dùng và mức nhân. Nêu ưu nhược điểm của mỗi loại.

Luồng ở mức người dùng	Luồng ở mức nhân
▶ Do trình ứng dụng tạo ra và quản lý ▶ Việc phân phối CPU được thực hiện cho cả tiến trình ▶ OS vẫn coi tiến trình như một đơn vị duy nhất với một trạng thái duy nhất	▶ Luồng mức nhân được HĐH tạo ra và quản lý ▶ OS cung cấp giao diện lập trình ▶ Cung cấp các lời gọi hệ thống mà trình ứng dụng có thể yêu cầu tạo, xóa luồng và thay đổi tham số liên quan quản lý dòng
▶ Ưu điểm: • Tiết kiệm thời gian do việc chuyển đổi luồng không đòi hỏi chuyển sang chế độ nhân • Trình ứng dụng có thể điều độ riêng, không phụ thuộc vào cách điều độ của OS • Có thể sử dụng khi OS không hỗ trợ đa luồng	▶ Ưu điểm: • Tăng tính đáp ứng và khả năng thực hiện đồng thời của các luồng trong cùng tiến trình ▶ HĐH Windows và Linux hỗ trợ luồng mức nhân
▶ Nhược điểm • Không tận dụng ưu điểm về tính đáp ứng của mô hình đa luồng • Không cho phép tận dụng kiến trúc nhiều CPU	▶ Nhược điểm • Tốc độ chậm do tạo và chuyển đổi luồng thực hiện trong chế độ nhân

Câu hỏi 1.9: Trình bày về điều độ quay vòng. Cho ví dụ minh họa về tính thời gian chờ đợi trung bình khi điều độ theo kiểu này.

Điều độ quay vòng (Round Robin)

- sửa đổi FCFS dùng cho các hệ chia sẻ thời gian, có thêm cơ chế phân phối lại bằng cách sử dụng ngắt của đồng hồ
- hệ thống xác định những khoảng thời gian nhỏ gọi là lượng tử/lát cắt thời gian t
- Tiến trình sẽ lần lượt được cấp CPU trong những khoảng thời gian t trước khi bị ngắt và CPU được cấp cho tiến trình khác
- khi cpu được giải phóng, hđh đặt thời gian của đồng hồ bằng độ dài lượng tử, lấy tiến trình ở đầu hàng đợi và cấp cpu cho nó
- tiến trình kết thúc trước khi hết thời gian t : trả quyền điều khiển cho hđh
- hết thời gian t mà tiến trình chưa kết thúc:
 - + đồng hồ sinh ngắt
 - + tiến trình đang thực hiện bị dừng lại
 - + quyền điều khiển chuyển cho hàm xử lý ngắt của hđh

+ hdh chuyển tiến trình về cuối hàng đợi, lấy tiến trình ở đầu và tiếp tục

=> cải thiện thời gian đáp ứng so với FCFS, thời gian chờ đợi trung bình vẫn dài

Ví dụ:

Tiến trình		Độ dài chu kỳ CPU	
P1		10	
P2		4	
P3		2	

2	4	6	8	10	12	14
2	2	2	2	2	2	2
P1	P2	P3	P1	P2	P1	P1

Thời gian chờ đợi của P1, P2, P3 lần lượt là 6, 6, và 4.

Thời gian chờ đợi trung bình = $(6 + 6 + 4)/3 = 5,33$.

Câu hỏi 1.10: Các thông tin nào được lưu trữ trong khối quản lý tiến trình PCB?

Thông tin về tiến trình hiện thời trong PCB được gọi là ngữ cảnh (context) của tiến trình.

Ngữ cảnh phải bao gồm những thông tin, Tùy từng trường hợp cụ thể, những thông tin này có thể khác nhau.

► TH đơn giản: Hệ thống chuyển sang thực hiện vào/ra rồi quay lại thực hiện tiếp tiến trình:

- Ngữ cảnh gồm thông tin có thể bị hàm xử lý ngắt thay đổi
- Nội dung thanh ghi, trạng thái CPU

► TH phức tạp: Sau khi ngắt, hệ thống thực hiện tiến trình khác

- Thay đổi trạng thái tiến trình
- Cập nhật thông tin thống kê trong PCB
- Chuyển PCB của tiến trình vào danh sách ứng với trạng thái mới
- Cập nhật PCB của tiến trình mới được chọn
- Cập nhật nội dung thanh ghi và trạng thái CPU

Câu hỏi 1.11: Trình bày về các tiêu chí đánh giá thuật toán điều độ tiến trình

- lượng tiến trình được thực hiện xong

+ số lượng tiến trình thực hiện xong trong 1 đơn vị thời gian

+ đo tính hiệu quả của hệ thống

- hiệu suất sử dụng CPU: cố gắng để CPU càng ít phải nghỉ càng tốt

- thời gian vòng đời trung bình của tiến trình: từ lúc có yêu cầu tạo tiến trình đến khi kết thúc

- thời gian chờ đợi

+ tổng thời gian tiến trình nằm trong trạng thái sẵn sàng và chờ cấp CPU

+ ảnh hưởng trực tiếp của thuật toán điều độ tiến trình

- thời gian đáp ứng: đây là tiêu chí hướng tới người dùng và thường được sử dụng trong hệ thống tương tác trực tiếp

- tính dự đoán được: vòng đời, thời gian chờ đợi, thời gian đáp ứng phải ổn định, không phụ thuộc vào tải của hệ thống

- tính công bằng: các tiến trình cùng độ ưu tiên phải được đối xử như nhau

Câu hỏi 1.12: Trình bày về điều độ đến trước phục vụ trước FCFS. Cho ví dụ minh họa về tính thời gian chờ đợi trung bình khi theo điều độ kiểu này

- Tiến trình yêu cầu CPU trước sẽ được cấp CPU trước.

- Hệ điều hành xếp tiến trình sẵn sàng vào hàng đợi FIFO

- Tiến trình mới được xếp vào cuối hàng đợi

- Tiến trình sở hữu CPU tới khi kết thúc hoặc chờ đợi vào ra

- Đặc điểm

+ Đơn giản, dễ thực hiện, đảm bảo tính công bằng

+ thời gian chờ trung bình lớn => ảnh hưởng tới hiệu suất chung của toàn hệ thống

Ví dụ: Cho 3 tiến trình với thứ tự xuất hiện và độ dài chu kỳ CPU như sau:

Tiến trình		Thời gian	
P1		24	
P2		3	
P3		3	
0		24	27
24		3	3
P1		P2	P3

Thời gian chờ đợi của P1, P2, P3 lần lượt là: 0, 24, 27.

Thời gian chờ đợi trung bình $(0 + 24 + 27)/3 = 17$

Câu hỏi 1.11: Thế nào là bế tắc ? Điều kiện xảy ra bế tắc là gì ?

Bế tắc là tình trạng một nhóm tiến trình có cạnh tranh về tài nguyên hay có hợp tác phải dừng vô hạn Do tiến trình phải chờ đợi một sự kiện chỉ có thể sinh ra bởi tiến trình khác cũng đang trong trạng thái chờ đợi.

Tình trạng bế tắc xảy ra khi 4 điều kiện sau đồng thời thỏa mãn:

► Loại trừ tương hỗ: có tài nguyên nguy hiểm, tại 1 thời điểm duy nhất 1 tiến trình sử dụng. Tiến trình khác yêu cầu tài nguyên này sẽ phải dừng lại chờ khi tài nguyên được giải phóng.

► Giữ và chờ: tiến trình giữ tài nguyên trong khi chờ đợi

► Không có phân phối lại (no preemption): tài nguyên do tiến trình giữ không thể phân phối lại cho tiến trình khác trừ khi tiến trình đang giữ tự nguyện giải phóng tài nguyên

► Chờ đợi vòng tròn: tồn tại nhóm tiến trình P1, P2, ..., Pn sao cho P1 chờ đợi tài nguyên do P2 đang giữ, P2 chờ tài nguyên do P3 đang giữ, ..., Pn chờ tài nguyên do P1 đang giữ.

Câu hỏi 1.13 : Thế nào là địa chỉ lô gic và địa chỉ vật lý ?

Địa chỉ logic:

• là địa chỉ được gán cho các lệnh và dữ liệu không phụ thuộc vào vị trí cụ thể tiến trình trong bộ nhớ. Được sinh ra trong tiến trình (CPU đưa ra)

- CTUD chỉ quan tâm tới địa chỉ logic còn địa chỉ vật lý là do HĐH.

Địa chỉ vật lý:

- Là địa chỉ chính xác trong bộ nhớ máy tính
- Các mạch nhớ sử dụng để truy nhập tới chương trình và dữ liệu

► Địa chỉ logic được chuyển thành địa chỉ vật lý nhờ khối ánh xạ địa chỉ. (MMU=Memory Mapping Unit)

Câu hỏi 1.14 : Trình bày kỹ thuật phân chương cố định bộ nhớ.

- Bộ nhớ được chia thành n phần

- Mỗi phần được gọi là một chương (partition)
- Mỗi chương ở một vị trí cố định
- Chương được sử dụng như một vùng nhớ độc lập

- Mỗi chương chứa được đúng một tiến trình

- Khi được tải vào, tiến trình được cấp phát một chương. Sau khi tiến trình kết thúc, HĐH giải phóng chương và chương có thể được cấp phát cho tiến trình mới.

- Chương có thể có kích thước bằng nhau hoặc khác nhau

+ Kích thước các chương bằng nhau:

- Ưu điểm: Đơn giản
- Nhược điểm: Kích thước chương trình > kích thước chương dẫn đến không thể cấp phát
- Gây phân mảnh trong

+ Kích thước các chương khác nhau: Có hai cách lựa chọn chương nhớ để cấp cho tiến trình đang chờ đợi bằng cách chọn chương có kích thước nhỏ nhất

- Mỗi chương có một hàng đợi riêng: tiến trình có kích thước phù hợp với chương nào sẽ nằm trong hàng đợi của chương đó

► Ưu điểm: Tiết kiệm bộ nhớ, giảm phân mảnh trong

► Nhược điểm: Hệ thống không tối ưu, có thời điểm hàng đợi chương lớn thì rỗng, hàng đợi chương nhỏ hơn chứa nhiều tiến trình.

- Một hàng đợi chung cho tất cả các chương: Mỗi khi có một chương trống tiến trình nằm gần đầu hàng đợi nhất và có kích thước phù hợp với chương nhất sẽ được tải và thực hiện

► Ưu điểm: Tiết kiệm bộ nhớ, giảm phân mảnh trong và tối ưu hệ thống

Nhận xét về phân chương cố định

► Ưu điểm

- Đơn giản và ít xử lý
- Giảm thời gian tìm kiếm

► Nhược điểm

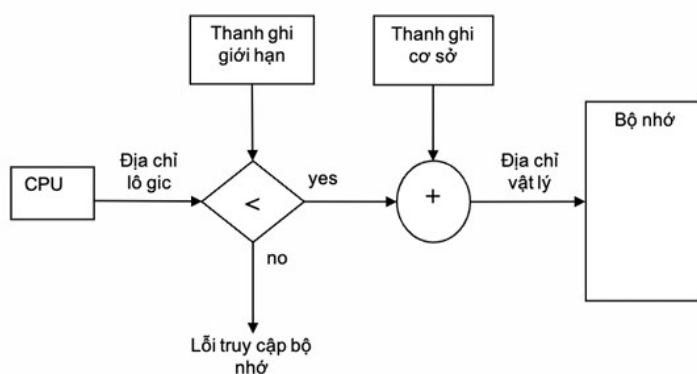
- Hệ số song song không thể vượt quá số lượng chương của bộ nhớ
- Bị phân đoạn bộ nhớ
 - Kích thước chương trình lớn hơn kích thước chương lớn nhất
 - Tổng bộ nhớ tự do còn lớn, nhưng không dùng để nạp các chương trình khác

Câu hỏi 1.15 : Trình bày cơ chế ánh xạ địa chỉ khi sử dụng kỹ thuật phân chương bộ nhớ.

Vị trí các chương thường không biết trước và có thể thay đổi

=> cần có cơ chế biến đổi địa chỉ logic thành vật lý

Ánh xạ địa chỉ do phần cứng đảm nhiệm



► Khi tiến trình được tải vào bộ nhớ, CPU dành 2 thanh ghi:

- Thanh ghi cơ sở: chứa địa chỉ bắt đầu của tiến trình
- Thanh ghi giới hạn: chứa độ dài chương

► Địa chỉ logic được so sánh với nội dung của thanh ghi giới hạn

- Nếu lớn hơn: lỗi truy cập
- Nhỏ hơn: đưa tới bộ cộng với thanh ghi cơ sở thành địa chỉ vật lý

► Nếu chương bị di chuyển thì nội dung của thanh ghi cơ sở bị thay đổi chứa địa chỉ mới

Câu hỏi 1.16: Trình bày về khái niệm phân đoạn bộ nhớ và ưu nhược điểm của phương pháp này

Phân đoạn bộ nhớ

- chương trình thường được chia thành nhiều phần: dữ liệu, lệnh, hàm, ngăn xếp
- phân đoạn bộ nhớ là chia chương trình thành các đoạn theo cấu trúc logic

- + đoạn chương trình: chứa toàn bộ mã chương trình, một số hàm hoặc thủ tục của chương trình
- + đoạn dữ liệu: chứa các biến toàn cục, các mảng
- + đoạn ngăn xếp: chứa ngăn xếp của tiến trình
- mỗi đoạn được phân vào 1 vùng nhớ, có kích thước không bằng nhau
- mỗi đoạn tương ứng với không gian địa chỉ riêng, được phân biệt bởi tên (STT) và độ dài của mình
- các vùng nhớ thuộc các đoạn khác nhau có thể nằm ở vị trí khác nhau

Ưu điểm:

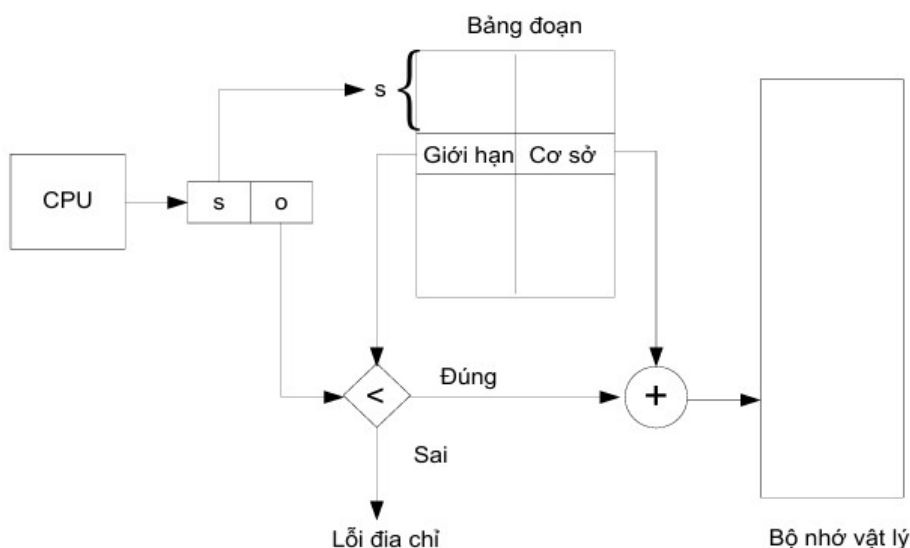
- Tránh hiện tượng phân mảnh trong, dễ sắp xếp bộ nhớ
- Dễ chia sẻ các đoạn giữa các tiến trình khác nhau
- Kích thước mỗi đoạn có thể thay đổi không ảnh hưởng đoạn khác

Nhược điểm:

- Có phân mảnh ngoài

Câu hỏi 1.17: Trình bày về cơ chế ánh xạ địa chỉ khi sử dụng kỹ thuật phân đoạn bộ nhớ

- Sử dụng bảng đoạn cho mỗi tiến trình. Mỗi ô tương ứng với 1 đoạn chứa
- + địa chỉ cơ sở: vị trí bắt đầu của đoạn trong bộ nhớ
- + địa chỉ giới hạn: độ dài đoạn, sử dụng để chống truy cập trái phép ra ngoài đoạn
- địa chỉ logic: < số thứ tự đoạn (s), độ dịch trong đoạn (o) >



- phần s trong địa chỉ được sử dụng để tìm tới ô thứ s trong bảng đoạn và truy cập hai giá trị giới hạn và cơ sở chứa trong ô này.
- So sánh độ dịch o với phần giới hạn chứa trong ô. Nếu o nhỏ hơn giới hạn thì đây là truy cập hợp lệ, ngược lại nếu $o \geq$ giới hạn thì địa chỉ này vượt ra ngoài phạm vi của đoạn $\Rightarrow$ lỗi truy cập.
- Trường hợp truy cập hợp lệ, độ dịch o được cộng với địa chỉ cơ sở để tạo ra địa chỉ vật lý.

Câu hỏi 1.18 : Việc định nghĩa và sử dụng khái niệm file đem lại những ưu điểm gì ? Khi đặt tên cho file cần quan tâm tới những quy định gì ?

Ưu điểm của việc định nghĩa và sử dụng khái niệm file

- Trừu tượng hóa: File giúp người dùng thao tác với bộ nhớ ngoài một cách đơn giản, không cần quan tâm đến cấu trúc vật lý phức tạp của thiết bị lưu trữ (đĩa cứng, đĩa mềm, v.v.).
- Tổ chức thông tin: File cho phép người dùng tổ chức thông tin một cách có cấu trúc, dễ quản lý và truy cập.
- Linh hoạt: File có thể chứa nhiều loại thông tin khác nhau (văn bản, chương trình, hình ảnh, v.v.) và được sử dụng cho nhiều mục đích khác nhau.
- Dễ sử dụng: Hệ điều hành cung cấp các thao tác cơ bản với file (tạo, xóa, đọc, ghi, v.v.) giúp người dùng dễ dàng sử dụng.

Quy định cần quan tâm khi đặt tên cho file:

- Độ dài tên file: Mỗi hệ điều hành có quy định khác nhau về độ dài tối đa của tên file.

Ví dụ:

MS-DOS giới hạn tên file ở 8 ký tự và 3 ký tự cho phần mở rộng.

Windows NTFS và Linux hỗ trợ tên file dài tới 255 ký tự.

- Ký tự hợp lệ: Một số ký tự không được phép xuất hiện trong tên file, tùy thuộc vào hệ điều hành.

Ví dụ:

Trong Windows, các ký tự như /, \, *, ?, | bị cấm.

Trong Linux, các ký tự như :, <, > cũng không được phép.

- Phân biệt chữ hoa và chữ thường: Một số hệ điều hành như Linux phân biệt chữ hoa và chữ thường trong tên file, trong khi Windows thì không.
- Phần mở rộng: Một số hệ điều hành yêu cầu hoặc khuyến khích sử dụng phần mở rộng để xác định loại file (ví dụ: .txt, .exe).

Câu hỏi 1.19 : Trình bày khái niệm thư mục ? Thông tin trong các khoản mục có nhất thiết phải lưu trữ gần nhau không ?

Thư mục (directory hoặc folder) là một cấu trúc được sử dụng trong hệ thống file để quản lý và tổ chức các file trên đĩa. Thư mục được tạo thành từ các khoản mục (entry), mỗi khoản mục đại diện cho một file. Khoản mục chứa thông tin của file: tên, kích thước, vị trí, kiểu file, . . . hoặc con trỏ tới nơi lưu trữ thông tin này.

Thông tin trong các khoản mục có nhất thiết phải lưu trữ gần nhau không ?

Không nhất thiết. Hệ thống file không yêu cầu các thông tin trong các khoản mục phải được lưu trữ gần nhau trên đĩa. Thông tin của một file có thể được phân bố tại nhiều vị trí khác nhau trên đĩa. Điều này giúp tận dụng không gian trống trên đĩa một cách hiệu quả và tránh lãng phí bộ nhớ.

Câu hỏi 1.20: Hệ điều hành có cần biết và hỗ trợ các kiểu cấu trúc file hay không? Vì sao?

- Đa số hệ điều hành không hỗ trợ và quản lý kiểu cấu trúc file.

- Cấu trúc file do chương trình ứng dụng và người dùng tự quản lý.
- Trong hệ điều hành UNIX, DOS, WINDOWS, file được xem như một tập hợp các byte.
- Các chương trình ứng dụng khác nhau sẽ tự tạo ra và quản lý cấu trúc file riêng của mình.

Ví dụ: chương trình đồ họa lưu file dưới dạng mã nhị phân đã được giải nén; chương trình hệ thống quản lý dữ liệu sẽ tạo ra file bao gồm các bản ghi.

Câu hỏi loại 2 điểm

Câu hỏi 2.1: Trình bày ngắn gọn về các thành phần cơ bản của hệ điều hành.

Các thành phần cơ bản của hệ điều hành bao gồm:

1. Quản lý tiến trình:

- * tạo và xóa tiến trình
- * tạm treo và khôi phục các tiến trình bị treo
- * Đồng bộ hóa các tiến trình.
- * tạo cơ chế liên lạc giữa các tiến trình
- * giải quyết bế tắc

2. Quản lý bộ nhớ:

- Cung cấp và giải phóng bộ nhớ theo yêu cầu của các tiến trình
- Quản lý không gian nhớ đã được cấp và không gian còn trống
- Quản lý việc phân phối bộ nhớ giữa các tiến trình
- Tạo ra bộ nhớ ảo và ánh xạ địa chỉ bộ nhớ ảo vào bộ nhớ thực

3. Quản lý vào/ra:

- Quản lý thông qua các chương trình điều khiển
- Đơn giản hoá và tăng hiệu quả quá trình trao đổi thông tin giữa các tiến trình với thiết bị vào

ra

4. Quản lý file và thư mục:

- Tạo, xóa file và thư mục
- Đọc ghi file
- Ánh xạ file và thư mục sang bộ nhớ ngoài

5. Hỗ trợ mạng và xử lý phân tán:

- Quản lý thiết bị mạng
- Hỗ trợ các giao thức truyền thông
- Quản lý truyền thông, cân bằng tải

6. Giao diện với người dùng:

- Giúp cho máy tính hiểu, xử lý được các chỉ thị, các lệnh của người dùng.

7. Các chương trình tiện ích và ứng dụng

Câu hỏi 2.2 : Trình bày về nhân của hệ điều hành ? Thế nào là chế độ nhân và chế độ người dùng ?

Nhân (kernel) là phần cốt lõi, thực hiện các chức năng cơ bản nhất, quan trọng nhất của HDH và thường xuyên được giữ trong bộ nhớ

- Kernel có nhiệm vụ quản lý tài nguyên hệ thống (liên lạc giữa các thành phần phần cứng và phần mềm)

* Máy tính hiện đại thường được thiết kế với hai chế độ thực hiện chương trình.

• **Nhân chạy trong chế độ đặc quyền— chế độ nhân:** là chế độ mà chương trình thực hiện trong đó có đầy đủ quyền truy cập và điều khiển phần cứng máy tính.

• **Chế độ người dùng:** chương trình thực hiện trong chế độ người dùng bị hạn chế rất nhiều quyền truy cập và sử dụng phần cứng.

► Việc phân biệt chế độ nhân và chế độ người dùng nhằm mục đích ngăn không cho CTUD vô tình hoặc cố ý thực hiện những thao tác làm ảnh hưởng tới hệ thống

Câu hỏi 2.3 : Trình bày về cấu trúc nguyên khối và cấu trúc phân lớp của hệ điều hành. Phân tích so sánh ưu nhược điểm hai kiểu cấu trúc này.

Câu hỏi 2.4: Trình bày về cấu trúc vi nhân của hệ điều hành. Phân tích so sánh cấu trúc này với cấu trúc nguyên khối và cấu trúc phân lớp.

Câu hỏi 2.6: Trình bày về cấu trúc phân lớp của hệ điều hành. So sánh ưu nhược điểm của cấu trúc này với cấu trúc nguyên khối và cấu trúc vi nhân.

Cấu trúc nguyên khối	Cấu trúc phân lớp	Cấu trúc vi nhân
• Toàn bộ chương trình và dữ liệu của HĐH có chung 1 không gian nhớ. Do vậy, có thể coi là một khối duy nhất. • HĐH trở thành một tập hợp các thủ tục hay các chương trình con	• Các thành phần được chia thành các lớp nằm chồng lên nhau • Mỗi lớp chỉ có thể liên lạc với lớp nằm kề bên trên và kề bên dưới • Mỗi lớp chỉ có thể sử dụng dịch vụ do lớp ngay bên dưới cung cấp	• Nhân có kích thước nhỏ, chỉ chứa các chức năng quan trọng nhất • Các chức năng còn lại được đặt vào các modul riêng: chạy trong chế độ đặc quyền hoặc người dùng. Khi có yêu cầu từ ứng dụng, nhân sẽ chuyển cho module tương ứng để xử lý và nhận lại kết quả, nhân chủ yếu đóng vai trò trung gian liên lạc.
• Ưu điểm: nhanh, không mất thời gian giữa các không gian nhớ	• Ưu điểm: chia nhỏ chức năng, dễ sử dụng, dễ sửa lỗi	• Ưu điểm: mềm dẻo, an toàn
• Nhược điểm: Không an toàn: khi bất kỳ thành phần nào có sự cố thì toàn bộ hệ thống sẽ không hoạt động đc; - Ko mềm dẻo và khó sửa đổi, thêm bớt tp nào sẽ ảnh hưởng tới toàn bộ hệ thống, khi có lỗi khó	• Nhược điểm: khó thiết kế (xác định số lớp, phân chia thành phần các chức năng của mỗi lớp), tốc độ chậm hơn cấu trúc nguyên khối	• Nhược điểm: tốc độ chậm hơn so với cấu trúc nguyên khối

xác định lỗi do tp nào gây ra		
-------------------------------	--	--

Câu hỏi 2.5: Trình bày về hệ điều hành đa chương trình. Lấy ví dụ minh họa để phân tích hiệu suất sử dụng CPU của hệ điều hành đa chương trình cao hơn so với hệ điều hành đơn chương trình.

- Hệ điều hành đa chương trình là hệ điều hành cho phép nhiều chương trình cùng được tải vào bộ nhớ và có thể thực hiện đồng thời.
- Khi một chương trình phải dừng lại để thực hiện vào ra, HDH sẽ chuyển CPU sang thực hiện một chương trình khác
- HDH đa chương trình phức tạp hơn rất nhiều so với HDH đơn chương trình

Ví dụ: Giả sử 3 chương trình cùng được tải vào bộ nhớ một lúc

- Đơn chương trình:

Chạy	Chờ đợi	Chạy	Chờ đợi	Chạy	Chờ đợi
Chương trình A		Chương trình B		Chương trình C	

Chương trình A, B, C được thực hiện tuần tự.

CPU sẽ phải chờ đợi trong thời gian các chương trình thực hiện I/O.

=> CPU bị nhàn rỗi trong khoảng thời gian dài.

- Đa chương trình

Chạy A	Chạy B	Chạy C	Chờ đợi	Chạy A	Chạy B	Chạy C	Chờ đợi
--------	--------	--------	---------	--------	--------	--------	---------

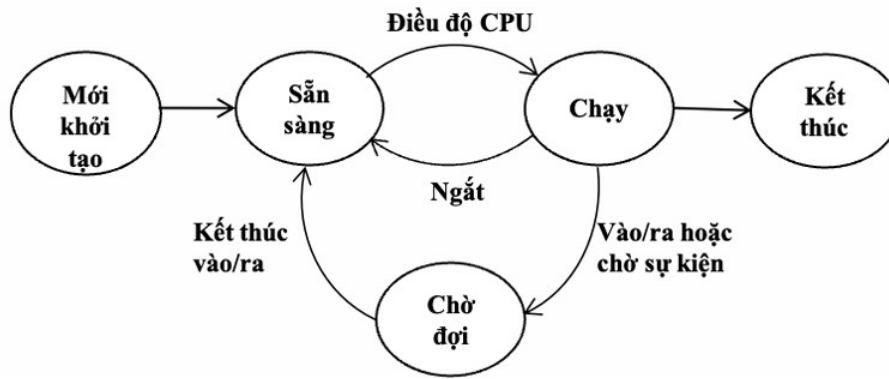
Thời gian thực hiện chương trình xen kẽ với thời gian chờ đợi vào/ra.

Thời gian chờ đợi của CPU trong chế độ đa chương trình giảm đáng kể so với trong trường hợp đơn chương trình

Câu hỏi 2.7 : Trình bày về năm trạng thái của tiến trình. Vẽ sơ đồ và giải thích về việc chuyển đổi giữa năm trạng thái này

- Trạng thái của tiến trình là một phần trong hoạt động hiện tại của tiến trình bao gồm 5 trạng thái:

- Mới khởi tạo (New): tiến trình đang được tạo ra
- Sẵn sàng (Ready): tiến trình chờ được cấp CPU để thực hiện lệnh
- Chạy (Running): lệnh của tiến trình được CPU thực hiện
- Chờ đợi (Waiting): tiến trình chờ đợi một sự kiện nào xảy ra (blocked)
- Kết thúc (Terminated): tiến trình đã kết thúc việc thực hiện nhưng vẫn chưa bị xóa



- ▶ Khởi tạo -> Sẵn sàng: tiến trình khởi tạo xong và đã được tải vào bộ nhớ, chỉ chờ được cấp CPU để chạy.
- ▶ Sẵn sàng -> Chạy: do kết quả điều độ CPU của OS, tiến trình được OS cấp phát CPU và chuyển sang trạng thái chạy
- ▶ Chạy -> Sẵn sàng: OS cấp phát CPU cho tiến trình khác, do kết quả điều độ/do ngắt xảy ra, tiến trình hiện thời chuyển sang trạng thái sẵn sàng và chờ được cấp CPU để chạy tiếp.
- ▶ Chạy -> Chờ đợi: Tiến trình chỉ chạy khi có 1 sự kiện nào đó xảy ra, chuyển sang trạng thái chờ được phân phối CPU để chạy tiếp.
- ▶ Chạy -> Kết thúc: khi tiến trình đã thực hiện

Câu hỏi 2.8 : Điều độ tiến trình là gì ? Điều độ dòng có khác điều độ tiến trình không ? Trình bày về điều độ có phân phối lại và không phân phối lại.

Điều độ (scheduling) hay lập lịch là quyết định tiến trình nào được sử dụng tài nguyên phần cứng khi nào, trong thời gian bao lâu.

Điều độ tiến trình và điều độ dòng:

- điều độ tiến trình: điều độ thực hiện với tiến trình, trong hệ thống trước kia tiến trình là đơn vị thực hiện chính, là đối tượng được cấp CPU
- điều độ dòng: Trong các hệ thống hỗ trợ luồng (thread), việc điều độ được thực hiện ở cấp độ luồng thay vì tiến trình. Luồng mức nhân là đơn vị thực hiện được HDH cấp CPU.

Điều độ có phân phối lại và không phân phối lại:

- Điều độ có phân phối lại (preemptive):
 - HDH có thể sử dụng cơ chế ngắt để thu hồi CPU của một tiến trình đang trong trạng thái chạy
- Điều độ không phân phối lại (nonpreemptive):
 - + Tiến trình đang ở trạng thái chạy sẽ được sử dụng CPU cho đến khi xảy ra một trong các tình huống sau:
 - ▶ Tiến trình kết thúc
 - ▶ Tiến trình phải chuyển sang trạng thái chờ đợi do thực hiện I/O
 - + Còn gọi là điều độ hợp tác: do chỉ thực hiện được khi tiến trình hợp tác và nhường CPU

+ Nếu tiến trình không hợp tác, dùng CPU vô hạn nên các tiến trình khác không được cấp CPU (Windows 96, NT)

- Ưu điểm của điều độ có phân phối lại:

+ So với điều độ không phân phối lại, điều độ có phân phối lại có nhiều ưu điểm hơn.

▶ HĐH chủ động hơn, không phụ thuộc vào hoạt động của tiến trình

▶ Đảm bảo chia sẻ thời gian thực sự

+ Tuy nhiên:

▶ Đòi hỏi phần cứng có bộ định thời gian và một số hỗ trợ khác

▶ Vấn đề quản lý tiến trình phức tạp hơn

Câu hỏi 2.7 : Trình bày hai biện pháp ngăn ngừa bế tắc (chọn 2 biện pháp bất kỳ trong số các biện pháp có thể).

Giữ và chờ: Có hai cách ngăn ngừa điều kiện này:

▶ Cách 1:

- Yêu cầu tiến trình phải nhận đủ toàn bộ tài nguyên cần thiết trước khi thực hiện tiếp
- Nếu không nhận đủ, tiến trình bị phong tỏa để chờ cho đến khi có thể nhận đủ tài nguyên

▶ Cách 2

- Tiến trình chỉ được yêu cầu tài nguyên nếu không giữ tài nguyên khác
- Trước khi yêu cầu thêm tài nguyên, tiến trình phải giải phóng tài nguyên đã được cấp và yêu cầu lại (nếu cần) cùng với tài nguyên mới

Chờ đợi vòng tròn: Xác định thứ tự cho các dạng tài nguyên và chỉ cho phép tiến trình yêu cầu tài nguyên sao cho tài nguyên mà tiến trình yêu cầu sau có thứ tự lớn hơn tài nguyên mà nó yêu cầu trước

Câu hỏi 2.9 : Trình bày một giải pháp giúp không xảy ra bế tắc khi sử dụng cờ hiệu cho bài toán triết gia ăn cơm.

Tóm tắt bài toán triết gia ăn cơm

- 5 triết gia ngồi trên ghế quanh 1 bàn tròn

+ trên bàn có 5 cái đũa: bên phải và bên trái mỗi người có 1 cái

+ triết gia có thể nhặt 2 chiếc đũa theo thứ tự bất kì: phải nhặt chiếc một và đũa không nằm trong tay người khác

+ khi cầm cả 2 đũa triết gia bắt đầu ăn và không đặt đũa trong thời gian ăn

+ sau khi ăn xong, triết gia đặt 2 đũa xuống bàn

Để tránh bế tắc có thể sử dụng giải pháp: Tại 1 thời điểm chỉ cho tối đa bốn người ngồi vào bàn

Giải pháp đồng bộ sử dụng thêm 1 cờ hiệu table có giá trị khởi tạo bằng 4. Triết gia phải gọi thao tác wait(table) trước khi ngồi vào bàn và lấy đũa.

Do giá trị cờ hiệu `table = 4` nên nếu đã có 4 người gọi `wait(table)` và chưa ai gọi `signal(table)` sau khi ăn xong thì người thứ năm gọi `wait(table)` sẽ bị phong tỏa.

Nhờ vậy, tại 1 thời điểm chỉ có tối đa 4 người có thể gọi các thao tác lấy đũa, tức là sẽ có ít nhất một người lấy được cả 2 đũa và vì thế không xảy ra bế tắc

```
semaphore chopstick[5] = {1,1,1,1,1};
```

```
semaphore table = 4;
```

```
void Philosopher(int i){    //tiến trình P(i)
    for(;;){    //lặp vô hạn
        wait(table);
        wait(chopstick[i]); //lấy đũa bên trái
        wait(chopstick[(i+1)%5]); //lấy đũa bên phải
        <Ăn cơm>
        (chopstick[(i+1)%5]);
        signal(chopstick[i]);
        signal(table);
        <Suy nghĩ>
    }
}

void main(){
// chạy đồng thời 5 tiến trình
    StartProcess(Philosopher(1));
    ...
    StartProcess(Philosopher (5));
}
```

Câu hỏi 2.10: Trình bày về thao tác và quá trình chuyển đổi giữa các tiến trình.

► Trong quá trình thực hiện, CPU có thể được chuyển từ tiến trình hiện thời sang thực hiện tiến trình khác. Thông tin về tiến trình hiện thời trong PCB được gọi là ngữ cảnh (context) của tiến trình.

► Việc chuyển đổi tiến trình xảy ra khi:

- Có ngắt: ngắt do đồng hồ/ngắt vào/ra
- Tiến trình gọi lời gọi hệ thống

► Khi chuyển sang thực hiện tiến trình khác, ngữ cảnh lưu vào PCB

► Khi được cấp phát CPU thực hiện trở lại, ngữ cảnh được khôi phục từ PCB vào các thanh ghi và bảng tương ứng.

Ngữ cảnh phải bao gồm những thông tin, Tùy từng trường hợp cụ thể, những thông tin này có thể khác nhau.

► TH đơn giản: Hệ thống chuyển sang thực hiện vào/ra rồi quay lại thực hiện tiếp tiến trình:

- Ngữ cảnh gồm thông tin có thể bị hàm xử lý ngắt thay đổi
- Nội dung thanh ghi, trạng thái CPU

► TH phức tạp: Sau khi ngắt, hệ thống thực hiện tiến trình khác

- Thay đổi trạng thái tiến trình
- Cập nhật thông tin thống kê trong PCB
- Chuyển PCB của tiến trình vào danh sách ứng với trạng thái mới
- Cập nhật PCB của tiến trình mới được chọn
- Cập nhật nội dung thanh ghi và trạng thái CPU

=> Chuyển đổi tiến trình đòi hỏi thời gian

Câu hỏi 2.12: Trình bày về giải pháp sử dụng lệnh máy Test_and_Set cho vấn đề loại trừ tương hỗ và đoạn nguy hiểm. Ưu nhược điểm

Sử dụng lệnh máy đặc biệt: Phần cứng được thiết kế có một số lệnh máy đặc biệt

- Hai thao tác kiểm tra giá trị và thay đổi giá trị cho một biến (một ô nhớ), hoặc các thao tác so sánh và hoán đổi giá trị hai biến, được thực hiện trong cùng một lệnh máy

=> đảm bảo được thực hiện cùng nhau mà không bị xen vào giữa – thao tác nguyên tử (atomic)

- gọi là lệnh “kiểm tra và xác lập” – Test_and_Set

Logic của lệnh Test_and_Set:

```
bool Test_and_Set(bool& val)
```

```
{  
    bool temp = val;  
    val = true;  
    return temp;  
}
```

- Ưu điểm

- đơn giản và nhanh chóng
- Có thể sử dụng cho trường hợp đa xử lý với nhiều CPU nhưng có bộ nhớ chung

- Nhược điểm:

- Chờ đợi tích cực
- Có thể gây đói tài nguyên (starvation)

Câu hỏi 2.13 : Trình bày kỹ thuật giúp tăng tốc độ truy cập bảng trang và bảng trang nhiều mức.

Kỹ thuật tăng tốc độ truy cập bảng trang

- Lưu bảng trang trong các thanh ghi: sử dụng một tập hợp các thanh ghi dành riêng để lưu bảng trang.

- Tốc độ truy cập rất cao
- Số lượng thanh ghi hạn chế => không áp dụng được

- Lưu bảng trang trong bộ nhớ trong.

- Vị trí mỗi bảng được trỏ bởi thanh ghi cơ sở bảng trang PTBR (Page Table Base Register)
- Đòi hỏi nhiều Thời gian để truy cập bảng

- Sử dụng bộ nhớ cache: Bộ nhớ đệm dịch địa chỉ (TLB) chứa một số lượng các bản ghi ánh xạ từ số thứ tự trang logic tới số thứ tự khung vật lý, giúp giảm số lần truy cập vào bảng trang trong bộ nhớ.

Bảng trang nhiều mức

- Chia bảng trang thành những phần nhỏ hơn
- Tổ chức bảng trang nhiều mức: Khoản mục của bảng mức trên chỉ tới bảng trang khác

Câu hỏi 2.14: Trình bày lý do phải đổi trang, và các bước tiến hành khi đổi trang.

Lý do phải đổi trang

- Khi xảy ra tình trạng thiếu trang: HĐH phải tìm ra một khung trống trong bộ nhớ đọc trang thiếu vào khung và tiến hành sau đó hoạt động bình thường.

- Bộ nhớ ảo > bộ nhớ thực và chế độ đa chương trình có lúc không còn khung nào trống để nạp trang mới

- Giải pháp đưa ra:

- Kết thúc tiến trình do không thỏa mãn nhu cầu bộ nhớ
- Trao đổi tiến trình ra đĩa và chờ thời điểm thuận lợi hơn mới nạp lại tiến trình vào bộ nhớ để thực hiện tiếp
- Sử dụng kỹ thuật đổi trang

Quá trình đổi trang:

- Bước 1: Xác định trang trên đĩa cần nạp vào MEM

- Bước 2: Nếu có khung trống trên MEM thì chuyển sang B4

- Bước 3:

- Lựa chọn 1 khung trên MEM để giải phóng, theo 1 thuật toán nào đó
- Ghi nội dung khung bị đổi ra đĩa (nếu cần), cập nhật bảng trang và bảng khung

- Bước 4: Đọc trang cần nạp vào khung vừa giải phóng; cập nhật bảng trang và bảng khung

- Bước 5: Thực hiện tiếp tiến trình từ điểm bị dừng trước khi đổi trang

Câu hỏi 2.15: Trình bày kỹ thuật đổi trang tối ưu và đổi trang vào trước ra trước.

Đổi trang tối ưu (OPT):

- ▶ Chọn trang sẽ không được dùng tới trong khoảng thời gian lâu nhất trong tương lai để đổi hay trang có lần sử dụng tiếp theo xa nhất
- ▶ Cho phép giảm tối thiểu sự thiếu trang và tối ưu theo tiêu chuẩn này
- ▶ HĐH đoán trước được nhu cầu sử dụng các trang trong tương lai
 - ⇒ Không áp dụng trong thực tế chỉ so sánh với các chiến lược khác

Đổi trang vào trước ra trước(FIFO)

Trang được đọc vào bộ nhớ trước sẽ bị đổi ra trước khi có yêu cầu đổi trang.

Trang bị trao đổi là trang nằm lâu nhất trong bộ nhớ

Khi trang được nạp vào bộ nhớ, số thứ tự trang được thêm vào cuối hàng đợi. Khi cần đổi trang, trang có số thứ tự ở đầu hàng đợi sẽ bị đổi.

Câu hỏi 2.16 : Trình bày các phương pháp xác định số lượng khung trang tối đa cấp cho mỗi tiến trình và xác định phạm vi cấp phát

Xác định số lượng khung trang tối đa cấp cho mỗi tiến trình

- Cấp phát số lượng khung cố định
 - + Cấp cho tiến trình một số lượng cố định khung để chứa các trang nhớ
 - + Số lượng được xác định vào thời điểm tạo mới tiến trình và không thay đổi trong quá trình tiến trình tồn tại
 - ▶ Cấp phát bằng nhau:
 - Các tiến trình được cấp số khung tối đa bằng nhau
 - Số lượng được xác định dựa vào kích thước MEM và mức độ đa chương trình mong muốn
 - ▶ Cấp phát không bằng nhau:
 - Các tiến trình được cấp số khung tối đa khác nhau
 - ▶ Cấp số khung tỉ lệ thuận với kích thước tiến trình
 - ▶ Có mức ưu tiên
- Cấp phát số lượng khung thay đổi
 - + Số lượng khung tối đa cấp cho mỗi tiến trình có thể thay đổi trong quá trình thực hiện
 - + Việc thay đổi phụ thuộc vào tình hình thực hiện của tiến trình
 - + Cho phép sử dụng bộ nhớ hiệu quả hơn phương pháp cố định
 - + Cần theo dõi và xử lý thông tin về tình hình sử dụng bộ nhớ của tiến trình

Xác định phạm vi cấp phát

- ▶ Cấp phát toàn thể:
 - Cho phép tiến trình đổi trang mới vào bất cứ khung nào (không bị khóa), kể cả khung đã được cấp phát cho tiến trình khác

► Cấp phát cục bộ:

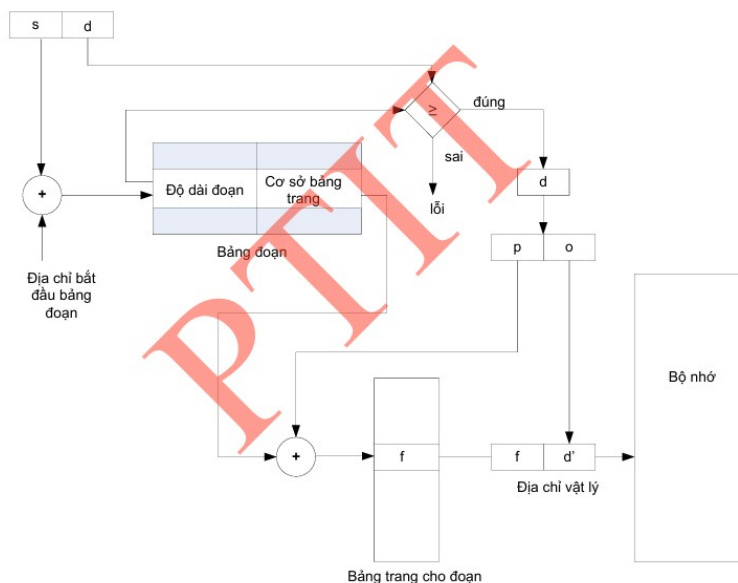
- Trang chỉ được đổi vào khung đang được cấp cho chính tiến trình đó

► Phạm vi cấp phát có quan hệ mật thiết với số lượng khung tối đa:

- Số lượng khung cố định tương ứng với phạm vi cấp phát cục bộ
- Số lượng khung thay đổi tương ứng với phạm vi cấp phát toàn thể

Câu hỏi 2.17: Trình bày phương pháp kết hợp phân trang và phân đoạn. Vẽ sơ đồ và giải thích cơ chế ánh xạ địa chỉ

- Phân đoạn chương trình, mỗi đoạn sẽ tiến hành phân trang
- Địa chỉ gồm: số thứ tự đoạn, số thứ tự trang, độ dịch trong trang
- Tiến trình có 1 bảng phân đoạn, mỗi đoạn có 1 bảng phân trang



Hình 3.15. Ánh xạ địa chỉ kết hợp phân đoạn với phân trang

Địa chỉ logic gồm ba phần (s,p,o). Phần thứ nhất s là số thứ tự đoạn. s cho phép xác định khoản mục ứng với đoạn trong bảng chia đoạn. Khoản mục chứa con trỏ tới bảng chia trang cho đoạn đó. Số thứ tự trang p cho phép xác định khung trang tương ứng. Độ dịch o sẽ được cộng vào địa chỉ khung để tính ra địa chỉ vật lý.

Câu hỏi 2.18: Trình bày về tình trạng trì trệ (thrashing)

- Trì trệ là tình trạng đổi trang liên tục do không đủ bộ nhớ
- Một tiến trình rơi vào tình trạng trì trệ khi thời gian đổi trang của tiến trình lớn hơn thời gian thực hiện
- Xảy ra khi:
 - Kích thước bộ nhớ hạn chế
 - Tiến trình đòi hỏi truy cập đồng thời nhiều trang nhớ
 - Hệ thống có mức độ đa chương trình cao

- Khi tiến trình rơi vào tình trạng trì trệ, tần suất thiếu trang tăng đáng kể => Sử dụng để phát hiện và giải quyết vấn đề trì trệ

- Kiểm soát tần suất thiếu trang

+ Theo dõi và ghi lại tần suất thiếu trang

+ Có thể đặt ra giới hạn trên và giới hạn dưới cho tần suất thiếu trang của tiến trình

- Tần suất vượt giới hạn trên:

- ▶ Cấp thêm cho tiến trình khung mới

- ▶ Nếu không thể tìm khung để cấp thêm, tiến trình sẽ bị treo hoặc bị kết thúc

- Tần suất thiếu trang thấp hơn giới hạn dưới: thu hồi một số khung của tiến trình

Câu hỏi 2.19: Trình bày về kỹ thuật sử dụng đệm trang. Sử dụng đệm trang đem lại những ưu điểm gì?

- HDH dành ra một số khung trống được kết nối thành danh sách liên kết gọi là các trang đệm

- Trang bị đổi như bình thường nhưng nội dung trang này không bị xóa ngay khỏi bộ nhớ

- Khung chứa trang được đánh dấu là khung trống và thêm vào cuối danh sách trang đệm

- Trang mới sẽ được nạp vào khung đứng đầu trong danh sách trang đệm

- Tới thời điểm thích hợp, hệ thống sẽ ghi nội dung các trang trong danh sách đệm ra đĩa

- Kỹ thuật đệm trang cho phép cải tiến tốc độ:

+ Nếu trang bị đổi có nội dung cần ghi ra đĩa, HDH vẫn có thể nạp trang mới vào ngay

- Việc ghi ra đĩa sẽ được lùi lại tới một thời điểm sau

- Thao tác ghi ra đĩa có thể thực hiện đồng thời với nhiều trang nằm trong danh sách được đánh dấu trống.

+ Trang bị đổi vẫn được giữ trong bộ nhớ một thời gian:

- Nếu có yêu cầu truy cập trong thời gian này, trang sẽ được lấy ra từ danh sách đệm và sử dụng ngay mà không cần nạp lại từ đĩa

- Vùng đệm đóng vai trò giống như bộ nhớ cache

Ưu điểm

- cải thiện tốc độ

- tối ưu hóa việc truy cập bộ nhớ

- giảm tần suất ghi đĩa

- tăng hiệu quả bộ nhớ

Câu hỏi 2.20 : Trình bày các cấu trúc dữ liệu dùng cho tổ chức bên trong của thư mục.

- Danh sách:

- Tổ chức thư mục dưới dạng danh sách các khoản mục

- Tìm kiếm khoản mục được thực hiện bằng cách duyệt lần lượt danh sách => Tìm kiếm trong danh sách chậm

- Thêm file mới vào thư mục:

- ▶ Duyệt cả thư mục để kiểm tra xem khoản mục với tên file như vậy đã có chưa

- ▶ Khoản mục mới được thêm vào cuối danh sách hoặc 1 ô trong bảng

- Mở file, xóa file

- Lưu trữ thư mục trong bộ nhớ

- Cây nhị phân:

- Tăng tốc độ tìm kiếm nhờ CTDL có hỗ trợ sắp xếp

- ví dụ: Hệ thống file NTFS của WinNT

- Bảng băm (hash table):

- Dùng hàm băm để tính vị trí của khoản mục trong thư mục theo tên file

- Thời gian tìm kiếm nhanh

- Hàm băm phụ thuộc vào kích thước của bảng băm => kích thước bảng cố định

- Tổ chức thư mục của DOS:

- Mỗi đĩa logic có cây thư mục riêng, bắt đầu từ thư mục gốc ROOT

- Thư mục gốc được đặt ở phần đầu của đĩa, ngay sau sector khởi động BOOT và bảng FAT

- Thư mục gốc chứa files và các thư mục con

- Thư mục con có thể chứa files và các thư mục cấp dưới nữa

- Được tổ chức dưới dạng bảng: mỗi khoản mục chiếm 1 dòng trong bảng và có kích thước cố định 32 bytes

- Tổ chức thư mục của Linux:

- Thư mục hệ thống file Ext2 của Linux có cách tổ chức đơn giản

- Khoản mục chứa tên file và địa chỉ I-node

- Thông tin còn lại về các thuộc tính file và vị trí các khối dữ liệu được lưu trên I-node chứ không phải thư mục

- Kích thước khoản mục phụ thuộc vào độ dài tên file

- Phần đầu của khoản mục có trường cho biết kích thước khoản mục

Câu hỏi 2.21: Trình bày cách kiểm soát truy cập file sử dụng mật khẩu và sử dụng danh sách quản lý truy cập

Kiểm soát truy cập file sử dụng mật khẩu

Mỗi file được gán một mật khẩu. Khi người dùng hoặc chương trình muốn truy cập file, hệ thống sẽ yêu cầu cung cấp mật khẩu.

=> Người dùng phải nhớ nhiều mật khẩu => Mỗi khi thao tác với tài nguyên lại gõ mật khẩu

Kiểm soát truy cập file sử dụng danh sách quản lý truy cập (ACL)

- Mỗi file được gán danh sách đi kèm, chứa thông tin định danh người dùng và các quyền người đó được thực hiện với file
- ACL thường được lưu trữ như thuộc tính của file/ thư mục
- Thường được sử dụng cùng với cơ chế đăng nhập
- Các quyền truy cập cơ bản:
 - ▶ Quyền đọc (r): người có quyền này được phép đọc nội dung file
 - ▶ Quyền ghi, thay đổi (w): được phép ghi vào file, tức là thay đổi nội dung file
 - ▶ Quyền xóa: được phép xóa file, quyền này tương đương với quyền thay đổi file
 - ▶ Quyền thay đổi chủ file (change owner)

Câu hỏi 2.22 : Trình bày các thao tác cơ bản với file. Phân tích rõ một hệ thống file có nhất thiết phải có thao tác mở file hay không.

- Tạo file: Tạo file trống chưa có dữ liệu; File được dành 1 chỗ trong thư mục kèm theo một số thuộc tính như thời gian tạo file, tên file, người tạo file
- Xóa file:
 - ▶ Giải phóng không gian mà dữ liệu của file chiếm trên đĩa
 - ▶ Giải phóng chỗ của file trong thư mục
- Mở file: Đọc các thuộc tính của file trên đĩa vào bộ nhớ để tăng tốc độ cho thao tác đọc ghi tiếp theo.
- Đóng file:
 - ▶ Xóa các thông tin về file ra khỏi bảng trong bộ nhớ để nhường chỗ cho các file sắp mở.
- Ghi vào file:
 - ▶ Dữ liệu được ghi vào vị trí hiện thời của file
 - ▶ Nếu vị trí hiện thời là cuối file, thông tin được thêm vào và kích thước file tăng lên
 - ▶ Nếu vị trí file không phải cuối file thông tin sẽ bị ghi đè lên
- Đọc file: Thông tin ở vị trí hiện thời sẽ được đọc
- Định vị (seek) Cho phép xác định vị trí hiện thời của file
- Đọc thuộc tính của file: Một số chương trình trong quá trình làm việc cần đọc các thuộc tính của file như quyền truy cập cho hệ thống bảo mật.
- Thay đổi thuộc tính của file: Đặt lại giá trị cho một số thuộc tính của file

- **Cắt bỏ nội dung file** : Thao tác xoá bỏ nội dung file và giải phóng vùng không gian cho file chiếm nhưng vẫn giữ lại các thuộc tính của file trừ độ dài và vị trí
- **Khoá file**: Khi đồng thời nhiều tiến trình thay đổi một nội dung file, có thể dẫn đến kết quả không mong đợi nên một số HĐH cho phép khoá file. Khi một tiến trình mở file, thì tiến trình đó có thể yêu cầu khoá file và không cho phép các tiến trình khác truy cập hoặc có thể đọc file nhưng không thể ghi vào file đó.

Thao tác mở file không bắt buộc khi thiết kế hệ thống file, nhưng nó rất thường được sử dụng để tăng hiệu quả thao tác vào-ra với file.

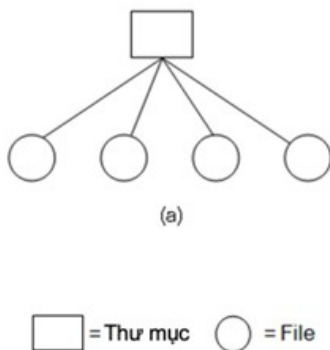
Khi mở file, hệ điều hành lưu thông tin về file vào bộ nhớ, giảm thiểu việc tìm kiếm lại trong thư mục mỗi lần truy cập.

Thao tác mở hỗ trợ quản lý quyền truy cập và giúp nhiều tiến trình đồng thời sử dụng file mà không gây xung đột. Mỗi file đang mở có thông tin như vị trí hiện thời và quyền truy cập, giúp thao tác đọc/ghi nhanh chóng và chính xác hơn.

Câu hỏi 2.23: Trình bày về cấu trúc hệ thống thư mục 1 mức và 2 mức

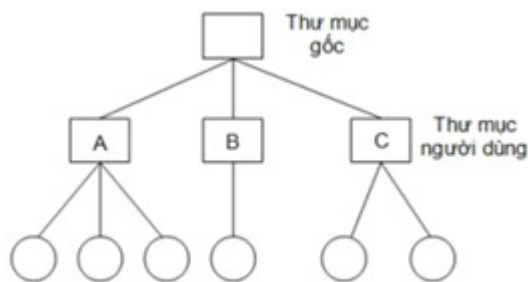
Thư mục 1 mức:

- Đơn giản nhất
- Có 1 thư mục duy nhất và tất cả các file được giữ trong thư mục này
- Khó chọn tên cho file
- Tìm kiếm file khó



Thư mục 2 mức:

- Phân cho mỗi người dùng 1 thư mục riêng (UFD: User File Directory), chứa các file của mình
- Khi người dùng truy cập file, file sẽ được tìm kiếm trong thư mục ứng với tên người đó
- Các người dùng khác nhau có thể đặt tên file trùng nhau
- Cô lập người dùng, Các file mà nhiều người dùng truy cập tới => chép vào từng thư mục của từng người dùng => lãng phí



Câu hỏi 2.24: Trình bày phương pháp cấp không gian cho file bằng các khối liên tiếp. Cho ví dụ minh họa. Ưu nhược điểm của phương pháp này là gì?

- Nguyên tắc: file được cấp phát các khối liên tiếp trên đĩa
- HĐH chọn 1 vùng trống có đủ số lượng khối cho file
- Bảng cấp phát xác định vị trí file gồm 1 khoản mục cho 1 file, khối bắt đầu, và số khối (độ dài) của file
- HĐH cấp phát trước và biết kích thước file khi tạo file

Ví dụ

Thư mục		
Tên file	Bắt đầu	Độ dài
fileA	1	2
fileB	3	3
fileB	8	5

Ưu điểm:

- + Cho phép truy cập trực tiếp và tuần tự
- + Đơn giản, tốc độ cao do tiết kiệm thời gian di chuyển đầu từ khi đọc các khối

Nhược điểm:

- Phải biết trước kích thước file khi tạo
- Khó tìm ra khoảng không gian trống đủ lớn trên đĩa để cấp phát cho file
- HĐH cần kiểm tra vùng trống khi cấp phát
- Gây phân mảnh ngoài

Câu hỏi 2.25: Trình bày về các phương pháp quản lý không gian trống trên đĩa: bảng bit, danh sách kết nối và danh sách vùng trống

Bảng bit

- Vector bit là mảng 1 chiều
 - Mỗi ô có kích thước 1 bit tương ứng với một khối trên đĩa
 - Khối được cấp phát có bit tương ứng là 0, khối trống: 1 hoặc ngược lại
- => Dễ tìm 1 hoặc nhóm các khối trống liên tiếp

- Với đĩa có kích thước lớn, đọc toàn bộ vector bit vào MEM có thể đòi hỏi khá nhiều không gian nhớ

Danh sách kết nối

- Các khối trống được liên kết với nhau thành danh sách
- Mỗi khối trống chứa con trỏ chỉ tới khối trống tiếp theo
- Địa chỉ khối trống đầu tiên được lưu ở vị trí đặc biệt trên đĩa và được HDH giữ trong MEM khi cần làm việc với các file
- Đòi hỏi truy cập lần lượt khi cần duyệt danh sách này
- HDH có thể cấp phát ngay các khối ở đầu danh sách

Danh sách vùng trống

- Các khối nằm liền nhau thường được cấp phát và giải phóng đồng thời
- Lưu vị trí khối trống đầu tiên của vùng các khối trống liên tiếp và số lượng các khối trống nằm liền sau đó
- Thông tin trên được lưu vào danh sách riêng

Câu hỏi loại 3 điểm

Câu hỏi 3.1: Trình bày khái niệm hệ điều hành. Phân tích rõ hai chức năng cơ bản của hệ điều hành.

Hệ điều hành là hệ thống phần mềm cơ bản, quan trọng và không thể thiếu trong một hệ thống máy tính, đóng vai trò trung gian giữa người sử dụng và phần cứng máy tính nhằm tạo ra môi trường giúp thực hiện các chương trình một cách thuận tiện.

Hai chức năng cơ bản của hệ điều hành.

- Quản lý tài nguyên
 - ▶ Đảm bảo cho tài nguyên hệ thống được sử dụng một cách có ích và hiệu quả
 - ▶ Các tài nguyên: bộ xử lý (CPU), bộ nhớ chính, bộ nhớ ngoài (các đĩa), các thiết bị vào ra
 - ▶ Phân phối tài nguyên cho các ứng dụng hiệu quả:
 - Yêu cầu tài nguyên được HDH thu nhận và đáp ứng bằng cách cấp cho chương trình các tài nguyên tương ứng
 - HDH cần lưu trữ tình trạng tài nguyên
 - ▶ Đảm bảo không xâm phạm tài nguyên cấp cho chương trình khác
 - ▶ Ví dụ: Lưu trữ thông tin trên đĩa HDH cần biết những vùng nào trên đĩa chưa được sử dụng để ghi thông tin lên những vùng này. Việc ghi thông tin cũng cần tính toán sao cho quá trình truy cập khi cần có thể thực hiện nhanh nhất.
- Quản lý việc thực hiện các chương trình

► Một chương trình đang trong quá trình chạy gọi là tiến trình (process). Chương trình cần được quản lý để thực hiện thuận lợi, tránh lỗi, đồng thời đảm bảo môi trường để việc xây dựng và thực hiện chương trình được thuận lợi.

► Để chạy chương trình cần thực hiện một số thao tác nhất định => HĐH giúp việc chạy chương trình dễ dàng hơn, người dùng không cần phải thực hiện thao tác

► Để tạo môi trường thuận lợi cho chương trình, HĐH tạo ra các máy ảo:

- Là máy logic với các tài nguyên ảo
- Tài nguyên ảo mô phỏng tài nguyên thực được thực hiện bằng phần mềm
- Cung cấp các dịch vụ cơ bản như tài nguyên thực
- Dễ sử dụng hơn, số lượng tài nguyên ảo có thể lớn hơn số lượng tài nguyên thực

Câu hỏi 3.2 : Dịch vụ của hệ điều hành là gì ? Trình bày những dịch vụ điển hình mà hệ điều hành cung cấp. Làm rõ về quá trình tải và chạy hệ điều hành khi mới khởi động.

Dịch vụ của hệ điều hành là các chức năng và tiện ích mà hệ điều hành cung cấp cho người dùng và các chương trình ứng dụng, giúp đơn giản hóa việc sử dụng và quản lý máy tính.

Những dịch vụ hệ điều hành cung cấp

- Tải và chạy chương trình:

• Để thực hiện, chương trình được tải từ đĩa vào bộ nhớ, sau đó được trao quyền thực hiện các lệnh. Khi thực hiện xong, cần giải phóng bộ nhớ và các tài nguyên

• Toàn bộ quá trình này tương đối phức tạp song lại diễn ra thường xuyên. HĐH sẽ thực hiện công việc phức tạp và lặp đi lặp lại này

• Do HĐH là chương trình đầu tiên được thực hiện khi khởi động hệ thống nên HĐH tự tải mình vào bộ nhớ

- Giao diện với người dùng: cho phép giao tiếp giữa HĐH và người dùng: Dưới dạng dòng lệnh (command-line), Giao diện đồ họa (Graphic User Interface-GUI)

- Thực hiện các thao tác vào/ ra dữ liệu

- Làm việc với hệ thống file: đọc, ghi, tạo, xóa, chép file hoặc làm việc với thư mục. Quản lý quyền truy cập, sao lưu.

- Phát hiện và xử lý lỗi: Đảm bảo cho hệ thống hoạt động ổn định, an toàn

- Truyền thông: Cung cấp dịch vụ cho phép thiết lập liên lạc và truyền thông tin

- Cấp phát tài nguyên

- Dịch vụ an ninh và bảo mật:

Tải và chạy hệ điều hành

Để hệ điều hành có thể hoạt động, nó cần được tải từ bộ nhớ ngoài (như ổ cứng) vào bộ nhớ trong (RAM). Quá trình này được gọi là khởi động (booting).

Hệ điều hành phải tự tải chính nó vào bộ nhớ trong. Do đó, cần có một chương trình nhỏ gọi là chương trình tải (bootloader) để thực hiện việc này.

Các bước tải và chạy hệ điều hành:

1. Lưu trữ chương trình tải: Chương trình tải được lưu trữ ở một vị trí xác định trên bộ nhớ ngoài (ví dụ: sector đầu tiên của ổ cứng).
2. BIOS kích hoạt chương trình tải: Khi máy tính khởi động, BIOS sẽ đọc và thực thi chương trình tải từ vị trí đã biết.
3. Chương trình tải nạp hệ điều hành: Chương trình tải sẽ nạp các thành phần cần thiết của hệ điều hành vào bộ nhớ trong.
4. Chuyển quyền điều khiển: Sau khi nạp xong, chương trình tải sẽ chuyển quyền điều khiển hệ thống cho hệ điều hành.

Câu hỏi 3.3 : Trình bày chi tiết về các thành phần của hệ điều hành.

1. Quản lý tiến trình:

- Tạo và xóa tiến trình: Bao gồm cả tiến trình người dùng và tiến trình hệ thống.
- Tạm treo và khôi phục các tiến trình bị treo.
- Đồng bộ hóa các tiến trình: Lập lịch cho các tiến trình, v.v.
- Tạo cơ chế liên lạc giữa các tiến trình.
- Giải quyết các bế tắc: Ví dụ như khi có xung đột về tài nguyên.
- Bế tắc: Là trạng thái khi chương trình cần tài nguyên nhưng phải chờ đợi mà không được cung cấp.

2. Quản lý bộ nhớ:

- Quản lý, cung cấp và giải phóng
- Cung cấp và giải phóng bộ nhớ theo yêu cầu của các tiến trình
- Quản lý không gian nhớ đã được cấp và không gian còn trống
- Quản lý việc phân phối bộ nhớ giữa các tiến trình => đảm bảo việc chạy song song giữa nhiều chương trình
- Tạo ra bộ nhớ ảo và ánh xạ địa chỉ bộ nhớ ảo vào bộ nhớ thực

3. Quản lý vào/ra:

- Quản lý thông qua các chương trình điều khiển
- Đơn giản hoá và tăng hiệu quả quá trình trao đổi thông tin giữa các tiến trình với thiết bị vào ra

4. Quản lý file và thư mục:

- Tạo, xóa file và thư mục
- Đọc ghi file
- Ánh xạ file và thư mục sang bộ nhớ ngoài

5. Hỗ trợ mạng và xử lý phân tán:

- Quản lý thiết bị mạng
- Hỗ trợ các giao thức truyền thông
- Quản lý truyền thông, cân bằng tải Thông qua các thành phần điều khiển, giao tiếp mạng.

6. Giao diện với người dùng:

- Đó là hệ thống thông dịch lệnh
- Giúp cho máy tính hiểu, xử lý được các chỉ thị, các lệnh của người dùng.
- Ví dụ: bash của Linux, command của window

7. Các chương trình tiện ích và ứng dụng

Câu hỏi 3.4 : Trình bày khái niệm dòng (thread) và mô hình đa dòng. Vấn đề sở hữu tài nguyên của tiến trình và dòng. Phân tích ưu điểm của mô hình đa dòng.

Mỗi đơn vị thực hiện lệnh của tiến trình, tức là 1 chuỗi lệnh được cấp phát CPU để thực hiện độc lập được gọi là **một luồng thực hiện**

Mô hình đa dòng là mô hình cho phép một tiến trình có nhiều luồng thực thi đồng thời, nhiều chuỗi lệnh, thực hiện nhiều nhiệm vụ cùng một lúc. Các thread trong cùng một tiến trình chia sẻ không gian địa chỉ và các tài nguyên của tiến trình đó, nhưng có các thanh ghi và ngăn xếp riêng.

Sở hữu tài nguyên

Tiến trình: Là đơn vị sở hữu tài nguyên cơ bản của hệ điều hành. Mỗi tiến trình được cấp phát một không gian địa chỉ riêng và các tài nguyên khác như file, thiết bị vào/ra.

Mỗi tiến trình và tất cả các luồng thuộc tiến trình đó sẽ sở hữu chung một số sở tài nguyên

- Không gian nhớ của tiến trình (logic): chứa chương trình (các lệnh), phần dữ liệu của tiến trình.
- Các tài nguyên khác: các file đang mở, thiết bị hoặc cổng vào/ra

Ưu điểm của đa dòng:

- ▶ Tăng hiệu năng và tiết kiệm thời gian
- ▶ Dễ dàng chia sẻ tài nguyên và thông tin
- ▶ Tăng tính đáp ứng
- ▶ Tận dụng được kiến trúc xử lý với nhiều CPU
- ▶ Thuận lợi cho việc tổ chức chương trình

Câu hỏi 3.5: Phân tích các vấn đề cần quan tâm trong sử dụng và quản lý tiến trình đồng thời (concurrent processes) đối với ba dạng tiến trình : tiến trình độc lập có cạnh tranh tài nguyên, tiến trình hợp tác nhờ chia sẻ tài nguyên, và tiến trình hợp tác nhờ trao đổi thông điệp.

Tiến trình độc lập có cạnh tranh tài nguyên

- ▶ Đảm bảo loại trừ tương hỗ (mutual exclusion):
 - Loại trừ tương hỗ là đảm bảo tiến trình này đang truy cập tài nguyên thì tiến trình khác không được truy cập tài nguyên đó

- Tài nguyên đó gọi là tài nguyên nguy hiểm. Đoạn chương trình có yêu cầu sử dụng tài nguyên nguy hiểm gọi là đoạn nguy hiểm (critical section)

- Hai tiến trình không được phép thực hiện đồng thời trong đoạn nguy hiểm của mình.

► Không để xảy ra bế tắc (deadlock):

- Bế tắc: tình trạng hai hoặc nhiều tiến trình không thể thực hiện tiếp do chờ đợi lẫn nhau tài nguyên mà không được cấp.

► Không để đói / thiếu tài nguyên (starvation)

- Đói tài nguyên (starvation) là chờ đợi quá lâu mà không đến lượt sử dụng một tài nguyên nào đó.

Tiến trình hợp tác với nhau qua tài nguyên chung: Trao đổi thông tin giữa các tiến trình hợp tác là chia sẻ vùng nhớ dùng chung (biến toàn thể), hay các file.

► Việc các tiến trình đồng thời truy cập dữ liệu dùng chung làm nảy sinh một số vấn đề :

- Đòi hỏi đảm bảo loại trừ tương hỗ

- Xuất hiện tình trạng bế tắc và đói

- Yêu cầu đảm bảo tính nhất quán dữ liệu

► Điều kiện chạy đua (race condition): tình huống mà một số luồng /tiến trình đọc, ghi dữ liệu sử dụng chung và kết quả phụ thuộc vào thứ tự các thao tác đọc, ghi

- Đặt thao tác truy cập và cập nhật dữ liệu dùng chung vào đoạn nguy hiểm

- Sử dụng loại trừ tương hỗ để các thao tác này không bị tiến trình khác xen ngang.

Tiến trình có liên lạc nhờ gửi thông điệp:

► Các tiến trình hợp tác có thể trao đổi thông tin trực tiếp với nhau bằng cách gửi thông điệp (message passing).

► Cơ chế liên lạc được hỗ trợ bởi thư viện của ngôn ngữ lập trình hoặc

► Không có yêu cầu loại trừ tương hỗ

► Có thể xuất hiện bế tắc và đói

Câu hỏi 3.6: Trình bày giải thuật Peterson cho đoạn nguy hiểm. Phân tích ưu nhược điểm của phương pháp này.

Giải thuật Peterson đề xuất cho bài toán đồng bộ hai tiến trình P0 và P1:

► P0 và P1 thực hiện đồng thời với một tài nguyên chung và một đoạn nguy hiểm chung

► Mỗi tiến trình thực hiện vô hạn và xen kẽ giữa đoạn nguy hiểm với phần còn lại của tiến trình

► Yêu cầu 2 tiến trình trao đổi thông tin qua 2 biến chung:

- Int Turn: xác định đến lượt tiến trình nào được vào đoạn nguy hiểm

- Cờ cho mỗi tiến trình: $flag[i]=true$ nếu tiến trình thứ i yêu cầu được vào đoạn nguy hiểm

Giải thuật Peterson được thể hiện như sau:

...

```

bool flag[2];
int turn;

void P0(){ //tiến trình P0
    for(;;){ //lặp vô hạn
        flag[0]=true;
        turn=1;
        while(flag[1] && turn==1);//lặp đến khi điều kiện không thỏa
        <Đoạn nguy hiểm>
        flag[0]=false;
        <Phần còn lại của tiến trình>
    }
}

void P1(){ //tiến trình P1
    for(;;){ //lặp vô hạn
        flag[1]=true;
        turn=0;
        while(flag[0] && turn==0);//lặp đến khi điều kiện không thỏa
        <Đoạn nguy hiểm>
        flag[1]=false;
        <Phần còn lại của tiến trình>
    }
}

void main(){
    flag[0]=flag[1]=false;
    turn=0;
    //tắt tiến trình chính, chạy đồng thời hai tiến trình P0 và P1
    StartProcess(P0);
    StartProcess(P1);
}

```

Ưu điểm:

Thỏa mãn các yêu cầu:

- Điều kiện loại trừ tương hỗ

► Điều kiện tiên tri:

- P0 chỉ có thể bị P1 ngăn cản vào đoạn nguy hiểm nếu $\text{flag}[1] = \text{true}$ và $\text{turn} = 1$ luôn đúng
- Có 2 khả năng với P1 ở ngoài đoạn nguy hiểm:
 - P1 chưa sẵn sàng vào đoạn nguy hiểm $\Rightarrow \text{flag}[1] = \text{false}$, P0 có thể vào ngay đoạn nguy hiểm
 - P1 đã đặt $\text{flag}[1] = \text{true}$ và đang trong vòng lặp `while` $\Rightarrow \text{turn} = 1$ hoặc $\text{turn} = 0$:
 - $\text{Turn} = 0$: P0 vào đoạn nguy hiểm ngay
 - $\text{Turn} = 1$: P1 vào đoạn nguy hiểm, sau đó đặt $\text{flag}[1] = \text{false}$ $\Rightarrow$ quay lại khả năng 1

► Chờ đợi có giới hạn

Nhược điểm:

- Sử dụng trên thực tế tương đối phức tạp
- Đòi hỏi tiến trình đang yêu cầu vào đoạn nguy hiểm phải nằm trong trạng thái chờ đợi tích cực
- Chờ đợi tích cực: tiến trình vẫn phải sử dụng CPU để kiểm tra xem có thể vào đoạn nguy hiểm? gây ra lãng phí CPU

Câu hỏi 3.6 : Trình bày phương pháp phát hiện và xử lý bế tắc bằng cách sử dụng đồ thị. Phân tích ưu điểm của phương pháp này so với phương pháp ngăn ngừa bế tắc.

Phát hiện bế tắc:

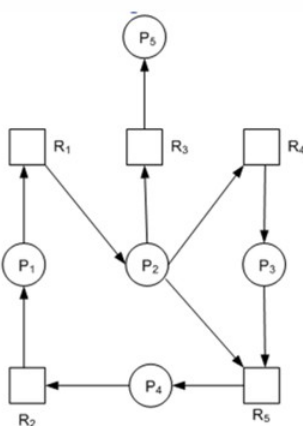
- Trường hợp mỗi dạng tài nguyên chỉ có một tài nguyên duy nhất $\Rightarrow$ sử dụng đồ thị biểu diễn quan hệ chờ đợi lẫn nhau giữa tiến trình

- Xây dựng đồ thị cấp phát tài nguyên:

+ các nút là tiến trình và tài nguyên

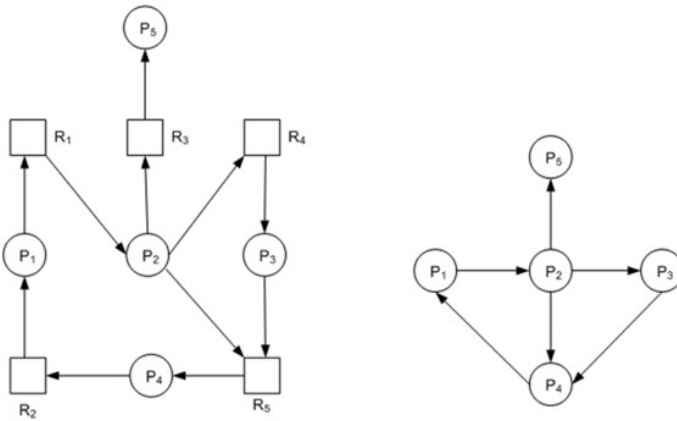
+ tài nguyên được nối với tiến trình bằng cung có hướng nếu tài nguyên được cấp cho tiến trình đó

+ tiến trình được nối với tài nguyên bằng cung có hướng nếu tài nguyên đang được cấp tài nguyên đó



Đồ thị chờ đợi:

- Được xây dựng từ đồ thị cấp phát tài nguyên bằng cách bỏ đi các nút tương ứng với tài nguyên và nhập các cung đi qua nút bị bỏ
- Cho phép phát hiện tình trạng tiến trình chờ đợi vòng tròn là điều kiện đủ để sinh ra bế tắc
- Sử dụng thuật toán phát hiện chu trình trên đồ thị có hướng để phát hiện bế tắc trên đồ thị chờ đợi.



Xử lý khi bị bế tắc

- Kết thúc tất cả tiến trình đang bị bế tắc
- Kết thúc lần lượt từng tiến trình đang bị bế tắc đến khi hết bế tắc:
 - HDH phải chạy lại thuật toán phát hiện bế tắc sau khi kết thúc 1 tiến trình
 - HDH có thể chọn thứ tự kết thúc tiến trình dựa trên tiêu chí nào đó
- Khôi phục tiến trình về thời điểm trước khi bị bế tắc sau đó cho các tiến trình thực hiện lại từ điểm này:
 - Đòi hỏi HDH lưu trữ trạng thái để có thể thực hiện quay lui và khôi phục về các điểm kiểm tra trước đó
 - Khi chạy lại, các tiến trình có thể lại rơi vào bế tắc tiếp
- Lần lượt thu hồi lại tài nguyên từ các tiến trình bế tắc cho tới khi hết bế tắc

Ưu điểm so với phương pháp ngăn ngừa bế tắc

Tiêu chí	Phương pháp sử dụng đồ thị	Phương pháp ngăn ngừa bế tắc
Linh hoạt cấp phát tài nguyên	Không ràng buộc thứ tự yêu cầu tài nguyên.	Phải tuân theo quy tắc nghiêm ngặt về thứ tự.
Hiệu suất hệ thống	Cao: Không cần kiểm tra quy tắc ngăn ngừa.	Thấp: Cần chi phí tính toán cao để ngăn ngừa bế tắc.
Khả năng xử lý bế tắc	Linh hoạt: Có thể xử lý bế tắc khi phát hiện.	Không có: Ngăn ngừa ngay từ đầu, không cần xử lý sau.
Đơn giản trong triển khai	Dễ dàng: Dễ triển khai trong hệ thống phức tạp.	Khó khăn: Cần các quy tắc phức tạp, khó kiểm soát.
Khả năng phục hồi	Hệ thống có thể phục hồi linh hoạt.	Không có bế tắc để phục hồi vì đã ngăn ngừa.
Tính toán trước	Không cần: Chỉ phát hiện khi bế tắc xảy ra.	Cần: Phải dự đoán tất cả tình huống bế tắc.

Câu hỏi 3.7 : Trình bày kỹ thuật tải trong quá trình thực hiện. Trình bày kỹ thuật liên kết động và thư viện dùng chung. Phân tích rõ ưu điểm mà từng phương pháp đem lại.

- Tải trong quá trình chạy

- + Hàm chưa bị gọi thì chưa tải vào bộ nhớ
- + Chương trình chính được load vào bộ nhớ và chạy
- + Khi có lời gọi hàm:

Chương trình sẽ kiểm tra hàm đó được tải vào chưa

Nếu chưa, chương trình sẽ tiến hành tải sau đó ánh xạ địa chỉ hàm vào không gian chung của chương trình và thay đổi bảng địa chỉ để ghi lại ánh xạ đó

- + Lập trình viên đảm nhiệm kiểm tra và tải hàm, HDH chỉ cung cấp các hàm thư viện cho module

- Liên kết động và thư viện dùng chung: Trong giai đoạn liên kết, không kết nối các hàm thư viện vào chương trình mà chỉ chèn các thông tin về hàm vào thư viện đó

+ Trong liên kết động, thay vì liên kết các hàm thư viện vào chương trình trong giai đoạn biên dịch, một đoạn mã nhỏ sẽ được chèn vào vị trí của hàm thư viện. Đoạn mã này chứa thông tin về mô-đun thư viện

+ Khi đoạn mã nhỏ được gọi, nó kiểm tra module tương ứng đã có trong bộ nhớ chưa. Nếu chưa, mô-đun thư viện sẽ được tải vào bộ nhớ.

+ Sau đó, địa chỉ của mô-đun sẽ được thay thế vào đoạn mã chèn, và trong các lần thực thi tiếp theo, mô-đun thư viện sẽ được sử dụng mà không cần tải lại.

+ Đối với mô-đun thư viện được sử dụng bởi nhiều tiến trình, tất cả tiến trình có thể dùng chung một bản duy nhất phần mã chương trình của thư viện. Thư viện khi đó được gọi là thư viện dùng chung.

Ưu điểm

Tải trong quá trình chạy	Liên kết động và thư viện dùng chung
- Tiết kiệm bộ nhớ vì chỉ tải những phần cần thiết. - Giảm thời gian khởi động chương trình, vì không cần tải toàn bộ chương trình.	- Tiết kiệm bộ nhớ: Các chương trình có thể chia sẻ các thư viện dùng chung - Cập nhật và bảo trì dễ dàng: Khi có thay đổi hoặc sửa lỗi trong thư viện, chương trình không cần phải biên dịch lại toàn bộ mã nguồn. Thay vào đó, chỉ cần thay đổi phiên bản thư viện và chương trình sẽ tự động sử dụng phiên bản mới. - Tiết kiệm không gian trên đĩa: Các thư viện chung chỉ cần có một bản sao duy nhất trong hệ thống, thay vì có nhiều bản sao cho mỗi chương trình.

Câu hỏi 3.8 : Trình bày kỹ thuật phân chương động bộ nhớ. Phân tích ưu nhược điểm của phương pháp này so với phân chương cố định. Lấy ví dụ minh họa cho các chiến lược cấp chương động mà hệ điều hành thường sử dụng: first fit, best fit, worst fit. Khi di chuyển chương sang vị trí khác cần thay đổi thông tin gì trong khối ánh xạ địa chỉ.

- ▶ Kích thước, số lượng và vị trí chương đều có thể thay đổi
- ▶ Các vùng trống bộ nhớ cũng có thể được liên kết thành một danh sách kết nối
- ▶ Khi một tiến trình yêu cầu bộ nhớ
 - HĐH cấp cho tiến trình 1 chương có kích thước đúng bằng tiến trình đó đang cần
 - Nếu tìm thấy vùng nhớ tương ứng
 - ▶ Vùng trống được chia thành 2 phần
 - ▶ Một phần cung cấp theo yêu cầu
 - ▶ Một phần trả lại danh sách vùng trống tự do
 - Nếu không tìm thấy
 - ▶ Phải chờ tới khi có được một vùng trống thỏa mãn
 - ▶ Cho phép tiến trình khác trong hàng đợi thực hiện (nếu độ ưu tiên đảm bảo)
- ▶ Khi tiến trình kết thúc sẽ tạo vùng trống trong Bộ nhớ và các vùng trống nằm cạnh nhau được nhập lại thành vùng lớn hơn

So với phân chương cố định

Tiêu chí	Phân chương động	Phân chương cố định
Linh hoạt trong cấp phát bộ nhớ	Cao: Kích thước phân chương thay đổi theo yêu cầu.	Thấp: Phân chương có kích thước cố định.
Sử dụng bộ nhớ	Tốt hơn: Tối ưu bộ nhớ, giảm lãng phí.	Kém hơn: Lãng phí nếu phân vùng quá lớn hoặc nhỏ.
Phân mảnh bộ nhớ	Tránh phân mảnh trong, gây phân mảnh ngoài	Tránh phân mảnh ngoài, gây phân mảnh trong

Ví dụ: trong bộ nhớ có 4 vùng trống có kích thước lần lượt như sau 3MB, 8MB, 7MB, và 10MB; yêu cầu cấp phát vùng nhớ kích thước 6MB. Ba chiến lược cấp phát ở trên sẽ cho kết quả như sau:

- Chiến lược first fit sẽ chọn khối 8MB để chia và cấp phát.
- Chiến lược best fit sẽ chọn vùng trống 7MB.
- Chiến lược worst fit sẽ chọn vùng trống 10MB.

Khi di chuyển chương sang vị trí khác, nội dung thanh ghi cơ sở sẽ được thay đổi thành địa chỉ mới của chương. Các phép ánh xạ sau đó vẫn diễn ra như cũ.

Câu hỏi 3.9 : Trình bày kỹ thuật phân đoạn bộ nhớ bao gồm cả cấu trúc và cách ánh xạ địa chỉ. Phân tích so sánh ưu nhược điểm của phân đoạn với phân trang.

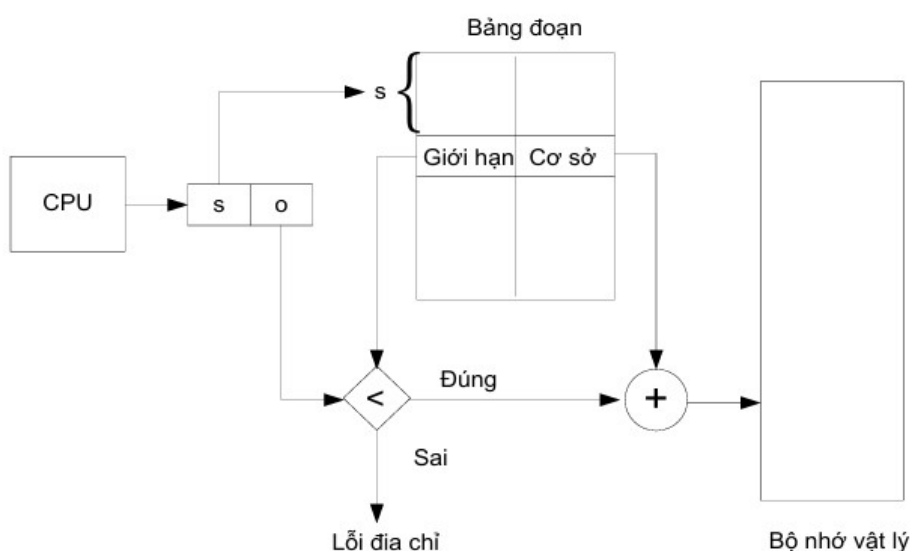
Phân đoạn bộ nhớ

- chương trình thường được chia thành nhiều phần: dữ liệu, lệnh, hàm, ngăn xếp

- phân đoạn bộ nhớ là chia chương trình thành các đoạn theo cấu trúc logic
- + đoạn chương trình: chứa toàn bộ mã chương trình, một số hàm hoặc thủ tục của chương trình
- + đoạn dữ liệu: chứa các biến toàn cục, các mảng
- + đoạn ngăn xếp: chứa ngăn xếp của tiến trình
- mỗi đoạn được phân vào 1 vùng nhớ, có kích thước không bằng nhau
- mỗi đoạn tương ứng với không gian địa chỉ riêng, được phân biệt bởi tên (STT) và độ dài của mình
- các vùng nhớ thuộc các đoạn khác nhau có thể nằm ở vị trí khác nhau

Ánh xạ địa chỉ

- Sử dụng bảng đoạn cho mỗi tiến trình. Mỗi ô tương ứng với 1 đoạn chứa
- + địa chỉ cơ sở: vị trí bắt đầu của đoạn trong bộ nhớ
- + địa chỉ giới hạn: độ dài đoạn, sử dụng để chống truy cập trái phép ra ngoài đoạn
- địa chỉ logic: < số thứ tự đoạn (s), độ dịch trong đoạn (o) >



- phần s trong địa chỉ được sử dụng để tìm tới ô thứ s trong bảng đoạn và truy cập hai giá trị giới hạn và cơ sở chứa trong ô này.
- So sánh độ dịch o với phần giới hạn chứa trong ô. Nếu o nhỏ hơn giới hạn thì đây là truy cập hợp lệ, ngược lại nếu $o \geq$ giới hạn thì địa chỉ này vượt ra ngoài phạm vi của đoạn => lỗi truy cập.
- Trường hợp truy cập hợp lệ, độ dịch o được cộng với địa chỉ cơ sở để tạo ra địa chỉ vật lý.

* Ưu điểm:

- Tránh hiện tượng phân mảnh trong, dễ sắp xếp bộ nhớ
- Dễ chia sẻ các đoạn giữa các tiến trình khác nhau
- Kích thước mỗi đoạn có thể thay đổi không ảnh hưởng đoạn khác

* Nhược điểm:

- Có phân mảnh ngoài

Ưu điểm so với phân trang

Phân đoạn bộ nhớ	Phân trang bộ nhớ
chia chương trình thành các đoạn theo cấu trúc logic	không quan tâm tới tổ chức logic và ý nghĩa của từng thành phần.
Cấp phát linh hoạt theo kích thước thực tế của đoạn.	Kích thước trang cố định, đều nhau
Không gây phân mảnh trong.	Tồn tại hiện tượng phân mảnh trong

Nhược điểm so với phân trang

Phân đoạn bộ nhớ	Phân trang bộ nhớ
Gây phân mảnh ngoài	Giảm phân mảnh ngoài
Quản lý các đoạn phức tạp hơn.	Quản lý đơn giản hơn với các trang cố định.

Câu hỏi 3.10 : Trình bày kỹ thuật nạp trang theo nhu cầu dùng cho bộ nhớ ảo. Nạp trang hoàn toàn theo nhu cầu khác với nạp trang trước như thế nào? Phân tích rõ cùng một lệnh có thể xảy ra nhiều sự kiện lỗi trang không.

Nạp trang theo nhu cầu

- Nạp trang theo nhu cầu dựa trên phân trang kết hợp trao đổi bộ nhớ-đĩa => Tiến trình được phân trang và các trang chứa trên đĩa
- Khi thực thi, nạp tiến trình vào MEM: chỉ nạp trang cần dùng => Các trang không nhất thiết nạp cùng một lúc
- Tiến trình gồm các trang trên đĩa và trong MEM: thêm bit P trong khoản mục bảng trang để phân biệt (P=1: đã nạp vào MEM)
- Quá trình kiểm tra và nạp trang: Tiến trình truy cập tới 1 trang, kiểm tra bit P. Nếu P=1, truy cập diễn ra bình thường. Nếu P=0, xảy ra sự kiện thiếu trang
- Ngắt xử lý thiếu trang:
 - Bước 1: HĐH tìm 1 khung trống trong MEM
 - Bước 2: Đọc trang bị thiếu vào khung trang vừa tìm được
 - Bước 3: Sửa lại khoản mục tương ứng trong bảng trang: đổi bit P=1 và số khung đã cấp cho trang
 - Bước 4: Khôi phục lại trạng thái của tiến trình và thực hiện tiếp lệnh bị ngắt

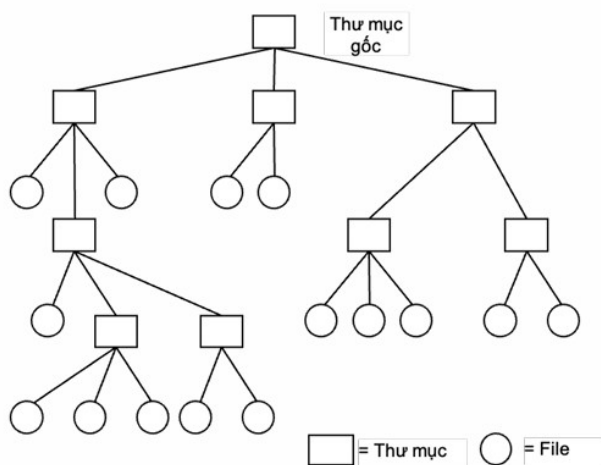
Nạp trang hoàn toàn theo nhu cầu	Nạp trang trước
• Bắt đầu một tiến trình mà không nạp bất kỳ trang nào vào bộ nhớ • Khi con trỏ lệnh được HĐH chuyển tới lệnh đầu tiên của tiến trình để thực hiện, sự kiện thiếu trang sẽ sinh ra và trang tương ứng được nạp vào • Tiến trình sau đó thực hiện bình thường cho tới lần thiếu trang tiếp theo	• Các trang chưa cần đến cũng được nạp vào bộ nhớ • Không hiệu quả

Cùng một lệnh có thể gây ra nhiều sự kiện lỗi trang nếu lệnh yêu cầu truy cập vào nhiều trang bộ nhớ chưa được nạp vào bộ nhớ. Tuy nhiên, hiện tượng này ít xảy ra trong thực tế do tính cục bộ của lệnh và dữ liệu, nơi các lệnh thường truy cập vào các vùng bộ nhớ gần nhau, giúp giảm thiểu sự thiếu trang.

Câu hỏi 3.11 : Trình bày cấu trúc thư mục dạng cây và dạng đồ thị không có chu trình. Cấu trúc thư mục dạng không chu trình có ưu điểm gì so với dạng cây ? Thế nào là đường dẫn tuyệt đối và đường dẫn tương đối.

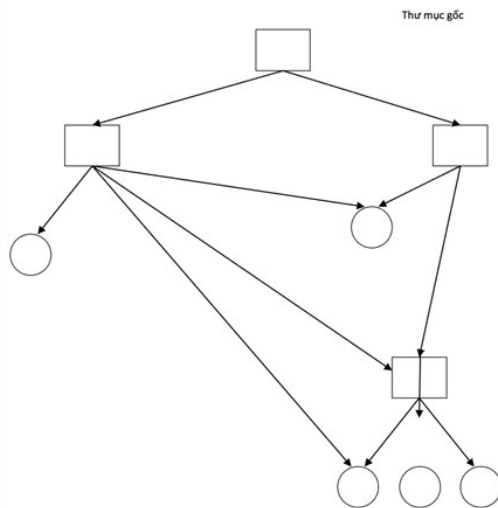
Cấu trúc thư mục dạng cây

- Thư mục con có thể chứa các thư mục con khác và các files
- Hệ thống thư mục được biểu diễn phần cấp như 1 cây: cành là thư mục, lá là file



- Phân biệt khoản mục file và khoản mục của thư mục con: thêm bit đặc biệt trong khoản mục
 - + 1: khoản mục của thư mục mức dưới
 - + 0: khoản mục của file
- Tại mỗi thời điểm, người dùng làm việc với thư mục hiện thời
- Tổ chức cây thư mục cho từng đĩa
 - + Trong hệ thống file như FAT của DOS, cây thư mục được xây cho từng đĩa. Hệ thống thư mục được coi là rừng, mỗi cây trên 1 đĩa
 - + Linux: toàn bộ hệ thống chỉ gồm 1 cây thư mục

Cấu trúc thư mục dạng đồ thị không có chu trình



- Chia sẻ files và thư mục để có thể xuất hiện ở nhiều thư mục riêng khác nhau
- Đây là 1 cấu trúc đồ thị không chứa chu trình, là mở rộng của cấu trúc cây
- Cây: lá và cành có thể đồng thời thuộc về những cành khác nhau
- Triển khai
 - + Sử dụng liên kết: con trỏ tới thư mục hoặc file khác
 - + Tạo bản sao của file và thư mục cần chia sẻ và chứa vào các thư mục khác nhau => phải đảm bảo tính đồng bộ và nhất quán => tránh sao lưu chồng chéo, thống kê 1 file nhiều lần

Cấu trúc thư mục dạng không chu trình có ưu điểm gì so với dạng cây ?

Linh hoạt, mềm dẻo hơn trong việc chia sẻ tài nguyên, tiết kiệm bộ nhớ và hỗ trợ quan hệ phức tạp giữa các thư mục

Đường dẫn tuyệt đối

- đường dẫn từ gốc của cây thư mục, đi qua các thư mục trung gian, dẫn tới file
- các thành phần của đường dẫn, tức là tên các thư mục ở giữa, được ngăn cách với nhau và với tên file bởi các ký tự đặc biệt

Đường dẫn tương đối

- đường dẫn tính từ thư mục hiện thời
- thêm 2 khoản mục đặc biệt cho thư mục “.” Để biểu diễn thư mục hiện tại, và “..” cho thư mục mức trên.

Câu hỏi 3.12 : Trình bày phương pháp cấp phát không gian cho file sử dụng danh sách kết nối và sử dụng khối chỉ số (I-node) (có ví dụ minh họa). Hai phương pháp này có điểm gì giống và khác nhau.

Danh sách kết nối

- Nguyên tắc: Các khối thuộc về mỗi file được kết nối với nhau thành danh sách kết nối, phần đầu khối chứa con trỏ trỏ tới khối tiếp theo
- File được cấp thêm khối mới, khối đó thêm vào cuối danh sách

- Các khối thuộc về 1 file có thể nằm ở vị trí bất kì trên đĩa
- Bảng cấp phát chứa con trỏ tới khối đầu tiên của file, HDH đọc lần lượt từng khối và sử dụng con trỏ để xác định khối tiếp theo
- Ưu điểm:
 - + Không bị phân mảnh do các khối không nhất thiết phải nằm cạnh nhau
 - + Không yêu cầu biết trước kích thước file
 - + Dễ tìm vị trí cho file, khoản mục đơn giản
- Nhược điểm:
 - + Không hỗ trợ truy cập trực tiếp
 - + Để đọc một khối ta phải đọc từ khối đầu tiên cho tới khối cần đọc
 - + Khi con trỏ thay đổi giá trị đổi, xác định khối nào thuộc file nào sẽ không chính xác giảm độ tin cậy, tính toàn vẹn Hệ thống file
 - + Tốc độ truy cập không cao

Ví dụ: một file bắt đầu từ khối 1

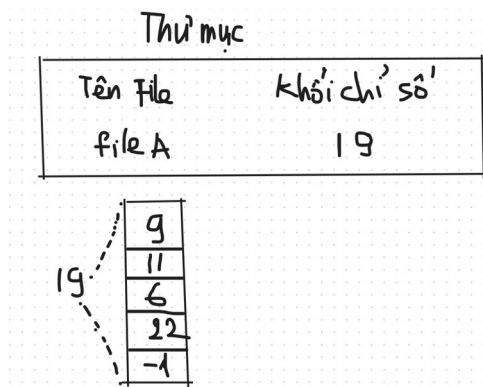
Khối	Con trỏ
1	3
3	8
8	6
6	-1 (kết thúc file)

Khối chỉ số (I-node)

Nguyên tắc: Mỗi file có một khối chỉ mục chính (index block) chứa danh sách các khối dữ liệu của file

- Tất cả con trỏ tới các khối thuộc về 1 file được tập trung 1 chỗ, trong một khối gọi là khối chỉ mục (I-node)
- Mảng chứa thuộc tính của file và vị trí các khối của file trên đĩa
- Ô thứ i của mảng chứa con trỏ tới khối thứ i của file
- Chọn kích thước I-node:
 - Nhỏ: tiết kiệm không gian nhưng không đủ con trỏ tới các khối nếu file lớn
 - Lớn: với file nhỏ chỉ chiếm 1 vài ô thì lãng phí
- Giải pháp:
 - Thay đổi kích thước i-node = sử dụng danh sách kết nối
 - Sử dụng I-node có cấu trúc nhiều mức

Ví dụ:



	Danh sách kết nối	Khối chỉ số
Giống nhau	- Điều để quản lý và cấp phát không gian trên đĩa cho files - sử dụng con trỏ để để trỏ đến các khối	
Khác nhau	- Các khối của file được liên kết với nhau qua các con trỏ trong từng khối. - không hỗ trợ truy cập trực tiếp => tốc độ truy cập không cao	- I-node chứa danh sách các con trỏ tới các khối của file. - cho phép truy cập trực tiếp => tốc độ truy cập cao

Câu hỏi 3.13 : Trình bày về yêu cầu phải đảm bảo tính toàn vẹn của hệ thống file và các phương pháp đảm bảo tính toàn vẹn

Yêu cầu phải đảm bảo tính toàn vẹn của hệ thống file

- Hệ thống file chứa nhiều CTDL có mối liên kết:
 - Thông tin về liên kết bị hư hại thì tính toàn vẹn của hệ thống bị phá vỡ
 - Liên kết đứt đoạn, không biết lưu ở khối nào tiếp theo
- Các khối không có mặt trong danh sách các khối trống, đồng thời cũng không có mặt trong một file nào
- Một khối có thể vừa thuộc về một file nào đó vừa có mặt trong danh sách khối trống
- HDH có các chương trình kiểm tra tính toàn vẹn của hệ thống file, được chạy khi hệ thống khởi động, đặc biệt là sau sự cố

Các phương pháp đảm bảo tính toàn vẹn

Đảm bảo tính toàn vẹn của hệ thống file sử dụng giao tác

- giao tác là 1 tập hợp các thao tác cần phải được thực hiện trọn vẹn cùng với nhau
- với hệ thống file: mỗi giao tác sẽ bao gồm những thao tác thay đổi liên kết cần thực hiện cùng nhau (ví dụ các thao tác cập nhật liên kết liên quan với 1 khối trên đĩa)
- Toàn bộ trạng thái hệ thống file được ghi lại trong file log
- Mỗi khi thực hiện giao tác, hệ thống kiểm tra xem giao tác có được thực hiện trọn vẹn không.
- Nếu giao tác không được thực hiện trọn vẹn, HDH sử dụng thông tin từ log để khôi phục hệ thống file về trạng thái không lỗi trước khi thực hiện giao tác

Câu hỏi 3.14: Trình bày phương pháp cấp phát không gian cho file sử dụng danh sách kết nối và sử dụng danh sách kết nối trên bảng chỉ số (có ví dụ minh họa) . So sánh sự giống nhau và khác nhau của hai phương pháp này.

Danh sách kết nối

- Nguyên tắc: Các khối thuộc về mỗi file được kết nối với nhau thành danh sách kết nối, phần đầu khối chứa con trỏ tới khối tiếp theo
- File được cấp thêm khối mới, khối đó thêm vào cuối danh sách
- Các khối thuộc về 1 file có thể nằm ở vị trí bất kì trên đĩa
- Bảng cấp phát chứa con trỏ tới khối đầu tiên của file, HDH đọc lần lượt từng khối và sử dụng con trỏ để xác định khối tiếp theo
- Ưu điểm:
 - + Không bị phân mảnh do các khối không nhất thiết phải nằm cạnh nhau
 - + Không yêu cầu biết trước kích thước file
 - + Dễ tìm vị trí cho file, khoản mục đơn giản
- Nhược điểm:
 - + Không hỗ trợ truy cập trực tiếp
 - + Để đọc một khối ta phải đọc từ khối đầu tiên cho tới khối cần đọc
 - + Khi con trỏ thay đổi giá trị đổi, xác định khối nào thuộc file nào sẽ không chính xác giảm độ tin cậy, tính toàn vẹn Hệ thống file
 - + Tốc độ truy cập không cao

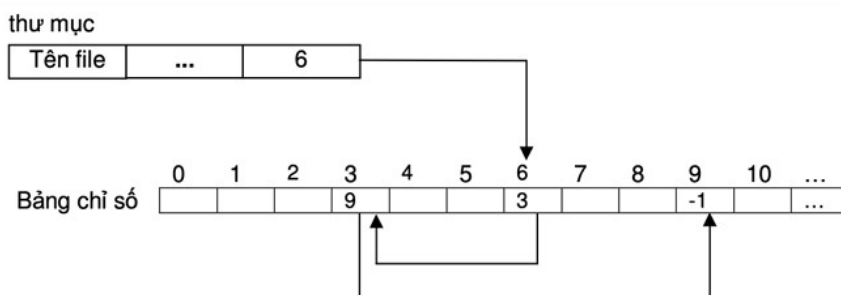
Ví dụ: một file bắt đầu từ khối 1

Khối	Con trỏ
1	3
3	8
8	6
6	-1 (kết thúc file)

Danh sách kết nối trên bảng chỉ số

- Bảng chỉ số: mỗi ô của bảng ứng với 1 khối của đĩa
- Con trỏ tới khối tiếp của file được chứa trong ô tương ứng của bảng
- Mỗi đĩa logic có 1 bảng chỉ số được lưu ở vị trí xác định
- Kích thước mỗi ô trên bảng phụ thuộc vào số lượng khối trên đĩa
- Cho phép truy cập file trực tiếp: đi theo chuỗi con trỏ chứa trong bảng chỉ mục
- Bảng FAT (File Allocation Table) được lưu ở đầu mỗi đĩa logic sau sector khởi động
- FAT12, FAT16, FAT32: Mỗi ô của bảng có kích thước 12, 16, 32 bit cho phép quản lý tối đa 212,216,232

Ví dụ:



	Danh sách kết nối	Danh sách kết nối trên Bảng chỉ số
Giống nhau	- Điều để quản lý và cấp phát không gian trên đĩa cho files - sử dụng con trỏ để để trỏ đến các khối	
Khác nhau	- Các khối của file được liên kết với nhau qua các con trỏ trong từng khối.	- Mỗi khối của file được chỉ định qua một ô trong bảng chỉ số.

Câu hỏi loại 4 điểm

Câu hỏi 4.1:

a) Trình bày các tiêu chí đánh giá thuật toán điều độ.

- lượng tiến trình được thực hiện xong
- + số lượng tiến trình thực hiện xong trong 1 đơn vị thời gian
- + đo tính hiệu quả của hệ thống
- hiệu suất sử dụng CPU: cố gắng để CPU càng ít phải nghỉ càng tốt
- thời gian vòng đời trung bình của tiến trình: từ lúc có yêu cầu tạo tiến trình đến khi kết thúc
- thời gian chờ đợi
- + tổng thời gian tiến trình nằm trong trạng thái sẵn sàng và chờ cấp CPU
- + ảnh hưởng trực tiếp của thuật toán điều độ tiến trình
- thời gian đáp ứng: đây là tiêu chí hướng tới người dùng và thường được sử dụng trong hệ thống tương tác trực tiếp
- tính dự đoán được: vòng đời, thời gian chờ đợi, thời gian đáp ứng phải ổn định, không phụ thuộc vào tải của hệ thống
- tính công bằng: các tiến trình cùng độ ưu tiên phải được đối xử như nhau

b) Trình bày thuật toán điều độ đến trước phục vụ trước và điều độ có mức ưu tiên.

Điều độ đến trước phục vụ trước (FCFS)	Điều độ có mức ưu tiên
- Tiến trình yêu cầu CPU trước sẽ được cấp CPU trước. - Hệ điều hành xếp tiến trình sẵn sàng vào hàng đợi FIFO - Tiến trình mới được xếp vào cuối hàng đợi - Tiến trình sở hữu CPU tới khi kết thúc hoặc chờ đợi vào ra	- Mỗi tiến trình có 1 mức ưu tiên - Tiến trình có mức ưu tiên cao hơn sẽ được cấp CPU trước - Các tiến trình có mức ưu tiên bằng nhau được điều độ theo FCFS - Mức ưu tiên được xác định theo nhiều tiêu chí khác nhau

- Đặc điểm + Đơn giản, dễ thực hiện, đảm bảo tính công bằng + thời gian chờ trung bình lớn => ảnh hưởng tới hiệu suất chung của toàn hệ thống + Thường là thuật toán điều độ ko phân phối lại	
---	--

Câu hỏi 4.2:

a) Trình bày thuật toán điều độ ưu tiên tiến trình ngắn nhất, thời gian còn lại ngắn nhất.

Điều độ ưu tiên tiến trình ngắn nhất (SPF)	Điều độ ưu tiên thời gian còn lại ngắn nhất (SRTF)
- Chọn trong hàng đợi tiến trình có chu kỳ sử dụng CPU tiếp theo ngắn nhất để phân phối CPU - Nếu có nhiều tiến trình với chu kỳ CPU tiếp theo bằng nhau, chọn tiến trình đứng trước - Thời gian chờ đợi trung bình nhỏ hơn nhiều so với FCFS - Khó thực hiện vì phải biết độ dài chu kỳ CPU tiếp: + Trong các hệ thống xử lý theo mẻ: dựa vào thời gian đăng kí tối đa do lập trình viên cung cấp + Dự đoán độ dài chu kỳ CPU tiếp theo: dựa trên độ dài TB các chu kỳ CPU trước đó - Không có phân phối lại	- SPF có thêm cơ chế phân phối lại (SRTF) - Khi 1 tiến trình mới xuất hiện trong hàng đợi, HDH so sánh thời gian còn lại của tiến trình đang chạy với thời gian còn lại của tiến trình mới xuất hiện - Nếu tiến trình mới xuất hiện có thời gian còn lại ngắn hơn, HDH thu hồi CPU của tiến trình đang chạy, phân phối cho tiến trình mới - Thời gian chờ đợi trung bình nhỏ - HDH phải dự đoán độ dài chu kỳ CPU của tiến trình - Việc chuyển đổi tiến trình ít hơn so với RR(quay vòng)

b) Điều độ theo mức ưu tiên có phân phối lại và không phân phối lại khác nhau thế nào ?

Điều độ theo mức ưu tiên không phân phối lại	Điều độ theo mức ưu tiên có phân phối lại
- Tiến trình đang chạy tiếp tục cho đến khi kết thúc hoặc tạm dừng chủ động - Không thu hồi CPU từ tiến trình đang chạy	- Tiến trình mới có thể lấy CPU ngay cả khi có tiến trình đang chạy - Thu hồi CPU nếu tiến trình mới có mức ưu tiên cao hơn

Câu hỏi 4.3:

a) Trình bày về các giải pháp phần cứng (cấm ngắt, sử dụng lệnh máy đặc biệt) cho vấn đề loại trừ tương hỗ và đoạn nguy hiểm.

Cấm ngắt

Trong trường hợp máy tính chỉ có 1 CPU, tại mỗi thời điểm chỉ có 1 tiến trình được thực hiện

- Tiến trình đang có CPU sẽ thực hiện cho đến khi tiến trình đó gọi dịch vụ hệ điều hành hoặc bị ngắt

- Để giải quyết Đoạn nguy hiểm => cấm không để xảy ra ngắt trong thời gian tiến trình đang ở trong đoạn nguy hiểm để truy cập tài nguyên

- Đảm bảo tiến trình được thực hiện trọn vẹn đoạn nguy hiểm và không bị tiến trình khác chen vào

=> đơn giản, giảm tính mềm dẻo của HDH => ảnh hưởng tới khả năng đáp ứng các sự kiện cần ngắt

Sử dụng lệnh máy đặc biệt: Phần cứng được thiết kế có một số lệnh máy đặc biệt

- Hai thao tác kiểm tra giá trị và thay đổi giá trị cho một biến (một ô nhớ), hoặc các thao tác so sánh và hoán đổi giá trị hai biến, được thực hiện trong cùng một lệnh máy

=> đảm bảo được thực hiện cùng nhau mà không bị xen vào giữa – thao tác nguyên tử (atomic)

- gọi là lệnh “kiểm tra và xác lập” – Test_and_Set

Logic của lệnh Test_and_Set:

```
bool Test_and_Set(bool& val)
```

```
{  
    bool temp = val;  
    val = true;  
    return temp;  
}
```

- Ưu điểm:

- Việc sử dụng tương đối đơn giản và trực quan
- Có thể dùng để đồng bộ nhiều tiến trình
- Có thể sử dụng cho trường hợp đa xử lý với nhiều CPU nhưng có bộ nhớ chung

- Nhược điểm:

- Chờ đợi tích cực
- Có thể gây đói tài nguyên (starvation)

b) Sử dụng Test_and_Set để thực hiện loại trừ tương hỗ cho bài toán các triết gia ăn cơm.

```
const int n = 5; // Số lượng tiến trình (5 triết gia)
```

```
bool lock[5]; // Mảng lock cho 5 triết gia
```

```
void P(int i) { // Tiến trình P(i)
```

```
    for(;;){ // Lặp vô hạn
```

```
        while (Test_and_Set(lock[i])) ; // lấy đũa bên trái
```

```
        while (Test_and_Set(lock[(i+1)%5])) ; // lấy đũa bên phải
```

```
        <Ăn Cơm>
```

```
        lock[i] = false; // trả đũa bên trái
```

```
        lock[(i+1)%5] = false; // trả đũa bên phải
```

```
        <Suy nghĩ>
```

```
    }
```

```
}
```

```
void main() {
```

```

    for (int i = 0; i < 5; i++)
        lock[i] = false;

// tắt tiến trình chính, chạy đồng thời 5 tiến trình

    StartProcess(P(1));

    ....

    StartProcess(P(5));

}

```

Sử dụng Test_and_Set là nguyên tử, thực hiện 2 bước kiểm tra và xác lập liên mạch, không bị gián đoạn đảm bảo rằng chỉ một triết gia có thể chiếm được mỗi chiếc đũa tại một thời điểm.

Giả sử hai triết gia (P1 và P2) cùng cố gắng lấy một chiếc đũa (lock).

Khi P1 gọi Test_and_Set(lock) trước: lock đang là false → Test_and_Set đặt lock = true.

Nếu ngay sau đó P2 cũng gọi Test_and_Set(lock): P2 thấy lock đã là true và phải chờ.

c) Phân tích rõ giải pháp sử dụng Test_and_Set sử dụng ở trên có thể gây bế tắc hoặc đói không.

- Bế tắc (Deadlock): Nếu tất cả các triết gia đều giữ đũa bên trái và chờ đũa bên phải, sẽ không ai có thể ăn => tạo thành vòng chờ đợi vô tận, khiến tất cả các tiến trình bị dừng lại.

- Đói (Starvation): Nếu một triết gia luôn bị các triết gia khác lấy đũa bên phải trước, họ sẽ không có cơ hội ăn, dẫn đến đói.

Câu hỏi 4.4:

a) Trình bày phương pháp sử dụng cờ hiệu (semaphore) cho vấn đề loại trừ tương hỗ và đoạn nguy hiểm.

Cờ hiệu semaphore S là 1 biến nguyên được khởi tạo bằng khả năng phục vụ đồng thời của tài nguyên. Giá trị của S chỉ có thể thay đổi nhờ gọi 2 thao tác là Wait và Signal:

- Wait(S):

+ giảm S đi 1 đơn vị

+ nếu giá trị của S < 0 thì tiến trình gọi wait(S) sẽ bị phong tỏa (blocked)

+ nếu giá trị của S không âm, tiến trình sẽ được thực hiện tiếp

- Signal(S):

+ tăng S lên 1 đơn vị

+ nếu giá trị của S ≤ 0, 1 trong các tiến trình đang bị phong tỏa được giải phóng và có thể thực hiện tiếp

- để tránh tình trạng chờ đợi tích cực, sử dụng 2 thao tác phong tỏa (block) và đánh thức (wakeup):

+ nếu tiến trình thực hiện thao tác wait, giá trị cờ hiệu âm thì nó sẽ bị phong tỏa (bởi thao tác block) và thêm vào hàng đợi của cờ hiệu

+ khi có 1 tiến trình thực hiện thao tác signal thì 1 trong các tiến trình bị khóa sẽ được chuyển sang trạng thái sẵn sàng nhờ thao tác đánh thức (wakeup)

- khi tiến trình cần truy cập tài nguyên, thực hiện thao tác Wait của cờ hiệu tương ứng

- sau khi dùng xong tài nguyên, tiến trình thực hiện thao tác Signal trên cùng cờ hiệu: tăng giá trị cờ hiệu và cho phép 1 tiến trình đang phong tỏa được thực hiện tiếp

b) Sử dụng cờ hiệu để thực hiện đồng bộ hóa cho bài toán Người sản xuất, người tiêu dùng với bộ đệm hạn chế.

Người sản xuất: tạo ra sản phẩm, xếp nó vào 1 chỗ gọi là bộ đệm, mỗi lần 1 sản phẩm

Người tiêu dùng: lấy sản phẩm từ bộ đệm, mỗi lần 1 sản phẩm

Dung lượng bộ đệm hạn chế, chứa tối đa N sản phẩm

- 3 yêu cầu đồng bộ:

+ người sản xuất và tiêu dùng không được sử dụng bộ đệm cùng lúc

+ khi bộ đệm rỗng, người tiêu dùng không nên cố lấy sản phẩm

+ khi bộ đệm đầy, người sản xuất không được thêm sản phẩm

- Giải quyết bằng cờ hiệu:

+ yêu cầu 1: sử dụng cờ hiệu lock khởi tạo bằng 1

+ yêu cầu 2: cờ hiệu empty, khởi tạo bằng 0

+ yêu cầu 3: cờ hiệu full, khởi tạo bằng N

const int N; //kích thước bộ đệm

semaphore lock = 1;

semaphore empty = 0;

semaphore full = N;

void producer () { //tiến trình người sản xuất

for (;;) {

<sản xuất >

wait (full);

wait (lock);

<thêm một sản phẩm vào bộ đệm>

signal (lock);

signal (empty);

}

}

void consumer () { //tiến trình người tiêu dùng

for (;;) {

wait (empty);

```

        wait (lock);
        <lấy một sản phẩm từ bộ đệm>
        signal (lock);
        signal (full);
        <tiêu dùng>
    }
}

void main(){
    StartProcess(producer);
    StartProcess(consumer);
}

```

Câu hỏi 4.5:

a) Trình bày giải pháp sử dụng monitor cho vấn đề loại trừ tương hỗ và đoạn nguy hiểm

- Đảm bảo loại trừ tương hỗ đối với đoạn nguy hiểm, đặt tài nguyên nguy hiểm vào trong monitor
- Tiến trình đang thực hiện trong monitor và bị dừng lại để đợi sự kiện hay một điều kiện nào đó được thỏa mãn => trả lại monitor để tiến trình khác sử dụng.
- Tiến trình chờ đợi sẽ được khôi phục lại từ điểm dừng sau khi điều kiện đang chờ đợi được thỏa mãn => Sử dụng các biến điều kiện
- Các biến điều kiện được khai báo và sử dụng trong monitor với 2 thao tác: cwait() và csignal():

+ x.cwait():

- ▶ Tiến trình đang ở trong monitor và gọi cwait bị phong tỏa cho tới khi điều kiện x xảy ra
- ▶ Tiến trình bị xếp vào hàng đợi của biến điều kiện x
- ▶ Monitor được giải phóng và một tiến trình khác sẽ được vào

+ x.csignal():

- ▶ Tiến trình gọi csignal để thông báo điều kiện x đã thỏa mãn
- ▶ Nếu có tiến trình đang bị phong tỏa và nằm trong hàng đợi của x do gọi x.cwait() trước đó sẽ được giải phóng
- ▶ Nếu không có tiến trình bị phong tỏa thì thao tác csignal sẽ không có tác dụng gì cả

b) Sử dụng monitor để thực hiện loại trừ tương hỗ cho bài toán Người sản xuất, người tiêu dùng với bộ đệm hạn chế.

- bộ đệm buffer được khai báo như dữ liệu cục bộ của monitor có thể truy cập bằng hai thao tác append (thêm vào) và take (lấy ra). Monitor sử dụng hai biến điều kiện notFull và notEmpty để tránh việc thêm sản phẩm vào bộ đệm đầy hoặc lấy sản phẩm khỏi bộ đệm rỗng.

```
monitor BoundedBuffer {
```



```

product buffer[N]; //bộ đệm chứa N sản phẩm kiểu product
int count;         //số lượng sản phẩm hiện thời trong bộ đệm
condition notFull, notEmpty; //các biến điều kiện

```

public:

```

BoundedBuffer( ) { //khởi tạo
    count = 0;
}

void append (product x) {
    if (count == N)
        notFull.cwait ( ); //dừng và chờ đến khi buffer có chỗ
    <Thêm một sản phẩm vào buffer>
    count++;
    notEmpty.csignal ( );
}

product take ( ) {
    if (count == 0)
        notEmpty.cwait ( ); //chờ đến khi buffer không rỗng
    <Lấy một sản phẩm x từ buffer>
    count--;
    notFull.csignal ( );
}
}

```

```

void producer ( ) { //tiến trình người sản xuất
    for (;;) {
        <Sản xuất sản phẩm x>
        BoundedBuffer.append (x);
    }
}

```

```

void consumer ( ) { //tiến trình người tiêu dùng
    for (;;) {
        product x = BoundedBuffer.take ( );
        <Tiêu dùng x>
    }
}

```

```
void main() {
    Thực hiện song song producer và consumer.
}
```

Câu hỏi 4.6:

a) Trình bày khái niệm phân trang bộ nhớ.

- Bộ nhớ vật lý được chia thành từng khối có kích thước bằng nhau: khung trang vật lý (page frame):
 - Khung trang vật lý được đánh số $0, 1, 2, \dots$: địa chỉ vật lý của khung trang
 - Khung trang được dùng làm đơn vị phân phối nhớ
- Không gian địa chỉ logic của tiến trình hay bộ nhớ logic (chương trình) được chia thành những khối gọi là trang (page) có kích thước bằng khung trang.
- Tiến trình được cấp các khung để chứa các trang của mình.
- Trang có thể chứa trong các khung nằm rải rác trong bộ nhớ
- Bảng trang: HĐH quản lý việc cấp phát khung cho mỗi tiến trình bằng bảng trang (Page table): mỗi ô tương ứng với 1 trang và chứa số khung cấp cho trang đó. Mỗi tiến trình có bảng trang riêng

b) Trình bày về ánh xạ địa chỉ khi phân trang bộ nhớ.

- Ánh xạ địa chỉ gồm 2 phần:
 - Số thứ tự trang (p)
 - Độ dịch (địa chỉ lệch) của địa chỉ so với đầu trang (o)
- Nếu kích thước trang là $2n$. Biểu diễn địa chỉ logic dưới dạng địa chỉ có độ dài $(m + n)$ bit
 - m bit cao: biểu diễn số thứ tự trang
 - n bit thấp: biểu diễn độ dịch trong trang nhớ
- Chuyển địa chỉ logic sang địa chỉ vật lý:
 - Lấy m bit cao của địa chỉ $\Rightarrow$ được số thứ tự trang
 - Dựa vào bảng trang, tìm được số thứ tự khung vật lý (k)
 - Địa chỉ vật lý bắt đầu của khung là $k * 2n$
 - Địa chỉ vật lý của byte được tham chiếu = địa chỉ bắt đầu của khung + Địa chỉ lệch (độ dịch)

Câu hỏi 4.6: Phòng tránh bế tắc là gì? Phân tích ưu điểm của việc phòng tránh bế tắc so với ngăn ngừa bế tắc

Trình bày phương pháp phòng tránh bế tắc sử dụng thuật toán người cho vay (banker's algorithm), cho ví dụ minh họa cụ thể.

Phòng tránh bế tắc:

- Cho phép 3 điều kiện đầu xảy ra và chỉ đảm bảo sao cho trạng thái bế tắc không bao giờ đạt tới

- Mỗi yêu cầu cấp tài nguyên của tiến trình sẽ được xem xét và quyết định tùy theo tình hình cụ thể
- HĐH yêu cầu tiến trình cung cấp thông tin về việc sử dụng tài nguyên (số lượng tối đa tài nguyên tiến trình cần sử dụng)

phòng tránh bế tắc	ngăn ngừa bế tắc
cung cấp tính linh hoạt và hiệu quả tài nguyên cao, nhưng yêu cầu tính toán phức tạp và dự đoán tình trạng hệ thống.	sử dụng quy tắc hay ràng buộc khi cấp phát tài nguyên => việc sử dụng tài nguyên kém hiệu quả, giảm hiệu năng của tiến trình.

Câu hỏi 4.7:

a) Trình bày chiến lược đổi trang ít sử dụng trong thời gian cuối.

- Trang bị đổi là trang mà thời gian từ lần truy cập cuối cùng đến thời điểm hiện tại là lâu nhất (lâu rồi không truy cập thì thay thế)
- Theo nguyên tắc cục bộ về thời gian, đó chính là trang ít có khả năng sử dụng tới nhất trong tương lai
- Thực tế LRU cho kết quả tốt gần như phương pháp đổi trang tối ưu
- Xác định trang có lần truy cập cuối diễn ra cách thời điểm hiện tại lâu nhất:

+ Sử dụng biến đếm:

- Mỗi khoản mục của bảng phân trang sẽ có thêm một trường chứa thời gian truy cập trang lần cuối - Là thời gian logic
- CPU cũng được thêm một bộ đếm thời gian logic này
- Chỉ số của bộ đếm tăng mỗi khi xảy ra truy cập bộ nhớ
- Mỗi khi một trang nhớ được truy cập, chỉ số của bộ đếm sẽ được ghi vào trường thời gian truy cập trong khoản mục của trang đó

=> Trường thời gian luôn chứa thời gian truy cập trang lần cuối

=> Trang bị đổi là trang có giá trị thời gian nhỏ nhất

+ Sử dụng ngăn xếp

- Ngăn xếp đặc biệt được sử dụng để chứa các số trang
- Mỗi khi một trang nhớ được truy cập, số trang sẽ được chuyển lên đỉnh ngăn xếp
- Đỉnh ngăn xếp sẽ chứa trang được truy cập gần đây nhất
- Đáy ngăn xếp chính là trang LRU, tức là trang cần trao đổi
- Tránh tìm kiếm trong bảng phân trang
- Thích hợp thực hiện bằng phần mềm

b) Trình bày chiến lược đổi trang sử dụng thuật toán đồng hồ.

Thuật toán đồng hồ cải tiến FIFO nhằm tránh thay những trang được nạp vào lâu vẫn có khả năng sử dụng

- Mỗi trang được gắn thêm 1 bit sử dụng U
 - Khi trang được truy cập đọc/ ghi: $U = 1$
 - Ngay khi trang được đọc vào bộ nhớ: $U = 1$
- Các khung/ trang có thể bị đổi được liên kết vào 1 danh sách vòng
- Khi một trang nào đó bị đổi, con trỏ được dịch chuyển để trỏ vào trang tiếp theo trong danh sách
- Khi có nhu cầu đổi trang, HDH kiểm tra trang đang bị trỏ tới
 - Nếu $U=0$: trang sẽ bị đổi ngay
 - Nếu $U=1$: đặt $U=0$ và trỏ sang trang tiếp theo, lặp lại quá trình
- U của tất cả các trang trong danh sách $=1$ con trỏ sẽ quay đúng 1 vòng, đặt U của các trang $=0$ và trang hiện thời bị trỏ sẽ bị đổi

Câu hỏi 4.8 :

a) Trình bày kỹ thuật phân chương sử dụng phương pháp kẻ cận (buddy).

Nguyên tắc: Chia đôi liên tiếp vùng trống tự do cho tới khi thu được vùng trống nhỏ nhất thỏa mãn

- Các chương và khối trống có kích thước là 2^k ($L \leq k \leq H$)

2^L : kích thước nhỏ nhất của chương

2^H : kích thước của toàn bộ không gian nhớ

- Yêu cầu cấp vùng nhớ S

+ $2^{H-1} < S \leq 2^H$: cấp cả 2^H

+ $S \leq 2^{H-1}$ Chia vùng trống tìm được thành 2 khối bằng nhau (gọi là buddies) 2^{H-1}

Nếu $2^{H-2} < S \leq 2^{H-1}$: cấp cả 2^{H-1}

Tiếp tục chia đôi tới khi tìm được vùng thỏa mãn $2^{k-1} < S \leq 2^k$

- Sau 1 thời gian, xuất hiện các khối trống có kích thước 2^k . HDH tạo ra danh sách kết nối, mỗi danh sách gồm các khối có kích thước bằng nhau.

- Các khối bị chia đôi được chuyển sang danh sách khối bé hơn. Ngược lại, khi có hai khối kẻ cận cùng kích thước, chúng sẽ được kết hợp lại thành khối lớn hơn.

- Khi yêu cầu cấp phát, hệ thống sẽ tìm trong danh sách các khối có kích thước phù hợp nhất. Nếu không có khối phù hợp, hệ thống sẽ tìm khối lớn hơn, chia nhỏ cho đến khi tìm được khối đủ lớn để cấp phát.

b) Phân tích rõ các điểm giống/khác nhau và ưu nhược điểm của phương pháp kẻ cận so với phân chương cố định và phân chương động (lưu ý không cần trình bày lại hai phương pháp sau)

Giống nhau:

- Phương pháp kẻ cận với phân chương cố định: chia bộ nhớ thành các phần nhỏ và độc lập.

- Phương pháp kề cận với phân chương động: cấp phát bộ nhớ cho tiến trình theo yêu cầu và không có kích thước cố định.

Khác nhau:

Phương pháp kề cận	Phân chương cố định	Phân chương động
Bộ nhớ chia thành khối có kích thước 2^k .	Bộ nhớ chia thành các chương với kích thước cố định.	Bộ nhớ chia thành các chương có kích thước thay đổi.
Cấp phát bộ nhớ bằng cách chia nhỏ hoặc kết hợp các khối trống.	Cấp phát một chương bộ nhớ cố định	Cấp phát bộ nhớ theo yêu cầu, không có kích thước cố định.
Các khối trống kề cận có thể được kết hợp lại thành một khối lớn hơn.	Không có cơ chế tự động kết hợp các vùng trống.	Vùng trống có thể tự động được hợp nhất khi tiến trình kết thúc.

Ưu điểm của phương pháp kề cận so với phân chương cố định và phân chương động

Phương pháp kề cận	Phân chương cố định	Phân chương động
Giảm phân mảnh bộ nhớ nhờ chia nhỏ và kết hợp các khối bộ nhớ.	Dễ gây phân mảnh trong	Dễ phân mảnh bộ nhớ ngoài
Linh hoạt hơn phân chương cố định nhờ chia nhỏ hoặc kết hợp khối trống.	Ít linh hoạt, bộ nhớ cố định cho mỗi tiến trình.	Linh hoạt quản lý phức tạp hơn.
Quản lý bộ nhớ trống hiệu quả		Quản lý hiệu quả nhưng dễ phân mảnh ngoài

Nhược điểm của phương pháp kề cận so với phân chương cố định và phân chương động

Phương pháp kề cận	Phân chương cố định	Phân chương động
Phức tạp hơn	Đơn giản, dễ triển khai	Dễ dàng hơn pp kề cận, nhưng phức tạp khi quản lý vùng trống
Tốn thời gian chia nhỏ và kết hợp	Cấp phát nhanh	Linh hoạt nhưng mất thời gian tìm vùng trống phù hợp

Câu hỏi 4.9:

a) Trình bày phương pháp cấp phát không gian cho file sử dụng các khối liên tiếp. Khi nào nên sử dụng phương pháp này cho hệ thống file ?

Cấp phát không gian cho file sử dụng các khối liên tiếp

- Nguyên tắc: file được cấp phát các khối liên tiếp trên đĩa
- HĐH chọn 1 vùng trống có đủ số lượng khối cho file
- Bảng cấp phát xác định vị trí file gồm 1 khoản mục cho 1 file, khối bắt đầu, và số khối (độ dài) của file
- HĐH cấp phát trước và biết kích thước file khi tạo file

Ưu điểm:

- + Cho phép truy cập trực tiếp và tuần tự
- + Đơn giản, tốc độ cao do tiết kiệm thời gian di chuyển đầu từ khi đọc các khối

Nhược điểm:

- Phải biết trước kích thước file khi tạo
- Khó tìm ra khoảng không gian trống đủ lớn trên đĩa để cấp phát cho file
- HĐH cần kiểm tra vùng trống khi cấp phát
- Gây phân mảnh ngoài

Khi nào nên sử dụng phương pháp này cho hệ thống file?

- kích thước file đã được biết trước
- file có kích thước ổn định
- sử dụng khi không gian trống trên đĩa không bị phân mảnh, hoặc khi phân mảnh có thể được kiểm soát dễ dàng.

=> khi hệ thống phải làm việc với các file có kích thước thay đổi liên tục hoặc có nhiều file nhỏ, phương pháp này có thể không tối ưu vì khó khăn trong việc tìm không gian trống liên tiếp trên đĩa.

b) Trình bày các bước cần thiết (trình bày bằng lời, không cần viết mã) để đọc bảng FAT từ thẻ nhớ USB vào bộ nhớ.

Bước 1: Xây dựng cấu trúc Boot

Bước 2: Đọc boot sector bằng hàm `absread(Driver = 4 (hoặc 5), 1, 0, &boot)`

Bước 3: Đọc FAT:

- > Khởi tạo vùng nhớ `fat = boot.FAT_size * boot.bytes_per_sector`
- > Số lượng sector của FAT là `n_sectors = boot.FAT_size`
- > Vị trí sector bắt đầu của FAT là `1_sector = boot.reserved`
- > Hàm đọc FAT: `absread(Driver = 4 (hoặc 5), n_sectors, 1_sector, root)`

c) Giả sử bảng FAT đã được đọc vào bộ nhớ tại địa chỉ « void *fat », viết đoạn chương trình trên C/C++ để liệt kê tất cả các cluster trống trong số N cluster đầu tiên. Giả sử một file bắt đầu tại cluster n, viết đoạn chương trình liệt kê các cluster thuộc về file đó.

// Liệt kê các cluster trống trong N cluster đầu tiên

```
for (int i = 0; i < N; i++){
    if (fat[i] == 0) print (" ", i);
}
```

// Liệt kê các cluster thuộc về file bắt đầu từ cluster thứ n

```
int cur = n;
while (cur < 0xFFFF)
{
    printf("->%d ", cur);
```

```

    cur = fat[cur];
}

```

Câu hỏi 4.10 :

a) Trình bày phương pháp sử dụng danh sách kết nối trên bảng chỉ số khi cấp phát không gian cho file.

- Bảng chỉ số: mỗi ô của bảng ứng với 1 khối của đĩa
- Con trỏ tới khối tiếp của file được chứa trong ô tương ứng của bảng
- Mỗi đĩa logic có 1 bảng chỉ số được lưu ở vị trí xác định
- Kích thước mỗi ô trên bảng phụ thuộc vào số lượng khối trên đĩa
- Cho phép truy cập file trực tiếp: đi theo chuỗi con trỏ chứa trong bảng chỉ mục
- Bảng FAT (File Allocation Table) được lưu ở đầu mỗi đĩa logic sau sector khởi động
- FAT12, FAT16, FAT32: Mỗi ô của bảng có kích thước 12, 16, 32 bit cho phép quản lý tối đa $2^{12}, 2^{16}, 2^{32}$

b) Trình bày các bước cần thiết (trình bày bằng lời, không cần viết mã) để đọc thư mục gốc từ thẻ nhớ USB vào bộ nhớ trong trường hợp hệ thống file của thẻ nhớ là FAT 16.

- + Bước 1: Xây dựng cấu trúc Boot và ROOT
- + Bước 2: Đọc boot sector bằng hàm absread(Driver = 4 (hoặc 5), 1, 0, &boot)
- + Bước 3: Đọc ROOT:
 - > Khởi tạo vùng nhớ root = boot.Root_size * 32
 - > Số lượng sector của ROOT là n_sectors = boot.Root_size * 32 / boot.bytes_per_sector
 - > Vị trí sector bắt đầu của ROOT là l_sector = boot.reserved + boot.FAT_size * boot.FAT_cnt
 - > Hàm đọc ROOT: absread(Driver = 4 (hoặc 5), n_sectors, l_sector, root)

c) Giả sử thư mục gốc của hệ thống file FAT 16 sử dụng tên file độ dài tối đa 8 ký tự đã được đọc vào bộ nhớ tại địa chỉ « void *root ». Viết đoạn chương trình trên C/C++ thực hiện hai việc : in tên và độ dài các file trong thư mục gốc, tìm một file có tên cho trước trong thư mục gốc và số thứ tự cluster đầu tiên của file đó